초급 교안

FastAPI 로 만드는 나의 첫 웹 서비스

웹을 처음 배우는 학생을 위한 — 작은 예제부터 미니 프로젝트까지

차례

0장 웹과 API 첫걸음

1장 첫 API 만들기

2장 데이터 보내기 — POST

3장 메모장 API — CRUD

4장 웹 페이지와 API

5장 메모장 웹 앱

6장 AI 맛보기

7장 미니 프로젝트

부록 용어 사전 · 명령어 모음 · 오류 해결 모음 · 스스로 더 공부하기

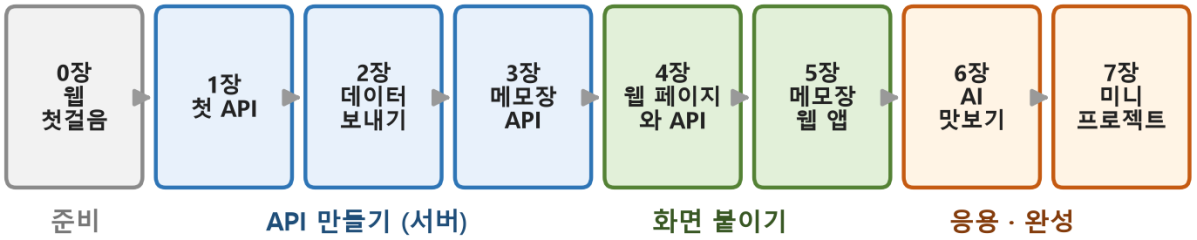
예제 코드 : github.com/sinaboro/ai-service-course → fastapi/초급과정

Python 3.12 · FastAPI 0.162 기준, 모든 실행 결과와 화면은 실제 실행한 것

강사 김대철

이 교안을 쓰는 방법

이 교안은 **웹을 처음 배우는 학생**이 FastAPI 로 작은 웹 서비스를 직접 만들어 보도록 짠 초급 교안이에요. 어려운 내용은 덜어 내고, **작은 예제를 하나씩 쌓아 올려** 마지막에는 나만의 웹 서비스를 완성해요.



작은 예제를 하나씩 쌓아 → 마지막에 나만의 웹 서비스를 만들어 발표해요

학습 지도 — API 만들기(1~3장) → 화면 붙이기(4~5장) → 응용과 프로젝트(6~7장)

이 교안의 약속

약속	뜻
예제는 짧게	예제 하나는 대부분 20~40줄이에요. 모든 줄에 한국어 주석이 있어요
브라우저로 확인	결과는 **브라우저 주소창**과 `/docs` **화면**으로 확인해요. 그림도 실제 화면이에요
한 번에 하나씩	장마다 새로 배우는 것은 한두 개뿐이고, 나머지는 앞에서 배운 것의 반복이에요
실제 결과만	교안의 실행 결과는 모두 실제로 실행한 결과예요 (시각 값은 실행할 때마다 달라요)

진행표 (예시)

회차	내용	시간
1	0장 웹과 API 첫걸음 — 요청과 응답, 환경 준비, 첫 서버	3시간
2	1장 첫 API 만들기 — 주소, 경로·쿼리, `/docs`	3시간
3	2장 데이터 보내기 — POST, Pydantic, 값 검사	3시간
4	3장 메모장 API — CRUD, 404 오류	3시간
5	4장 웹 페이지와 API — HTML, fetch, 동기·비동기, POST	3시간
6	5장 메모장 웹 앱 — 서버와 화면 합치기	3시간
7	6장 AI 맛보기 — AI 함수로 예측 서비스	2시간
8~9	7장 미니 프로젝트 — 기획, 만들기, 친구 폰 접속, 발표	6시간

알아 두기 | 준비물

- Python 3.10 이상 (권장 3.12), VS Code, 웹 브라우저(Chrome · Edge)
- 설치할 패키지는 `fastapi`, `uvicorn` 두 개뿐이에요 (0장 6절)
- 예제 코드 : `github.com/sinaboro/ai-service-course` → `fastapi/초급과정` 폴더. 장마다 폴더가 있고, 연습문제 정답은 각 장의 `연습문제_정답` 폴더에 있어요
- 파이썬 기초(변수 · 함수 · 리스트 · 딕셔너리 · if · for)를 안다고 가정해요

도움말 | 예제 실행 — 이것만 기억하세요

- **그 예제가 있는 폴더로** `cd` **한 뒤** 실행해요. (다른 폴더에서 켜면 `Could not import module` 오류)
- 서버 파일: `uvicorn 파일이름:app --reload` 로 켜고 → 브라우저로 확인 → `Ctrl + C` 로 끄기
- 보통 파이썬 파일(0장, 6장 일부): `python 파일이름.py`

목차

이 교안을 쓰는 방법 2

목차 4

0장 웹과 API 첫걸음 7

1. 웹은 어떻게 움직일까? . . . 7
2. URL(주소) 읽는 법 . . . 8
3. HTTP 메서드와 상태 코드 . . . 9
4. JSON 과 파이썬 딕셔너리 . . . 9
5. FastAPI 코드를 읽기 위한 파이썬 문법 . . . 10
6. 실습 환경 준비 — 폴더 · 가상환경 · 설치 . . . 12
7. 첫 서버 켜고 브라우저로 접속하기 . . . 13
8. 자주 막히는 곳과 정리 . . . 14

연습문제 . . . 14

연습문제 정답 및 해설 . . . 15

1장 첫 API 만들기 16

1. 주소(창구)를 여러 개 만들기 . . . 16
2. 경로 매개변수 — 주소 안에 값 넣기 . . . 17
3. 쿼리 매개변수 — 주소 뒤 ? 에 값 붙이기 . . . 19
4. /docs — 자동으로 생기는 API 시험 화면 . . . 20
5. (한 번만 보고 넘어가요) 코드로 요청 보내기 . . . 21
6. 자주 막히는 곳과 정리 . . . 22

연습문제 . . . 22

연습문제 정답 및 해설 . . . 23

2장 데이터 보내기 — POST 25

1. GET 과 POST — 보내는 방법이 달라요 . . . 25
2. POST 로 데이터 받기 — 회원 가입 . . . 26
3. Field 로 규칙 정하기 — 이상한 값 막기 . . . 27
4. 받은 값으로 계산하기 — 커피 주문 . . . 29
5. 자주 막히는 곳과 정리 . . . 31

연습문제 . . . 32

연습문제 정답 및 해설 . . . 33

3장 메모장 API — CRUD 35

1. CRUD — 거의 모든 웹 서비스의 뼈대 . . . 35
2. 메모장 API 만들기 . . . 35

3. 없는 메모는 404 — HTTPException . . . 37
4. /docs 에서 차례로 시험하기 . . . 37
5. 자주 막히는 곳과 정리 . . . 39
연습문제 . . . 39
연습문제 정답 및 해설 . . . 40

4장 웹 페이지와 API 42

1. 웹 페이지와 API 는 어떻게 함께 일할까? . . . 42
2. 1단계 · HTML 페이지 돌려주기 . . . 43
3. 2단계 · 버튼으로 API 부르기 — fetch 와 동기 · 비동기 . . . 45
4. 3단계 · 입력 칸으로 POST 보내기 . . . 48
5. 정리 . . . 51
연습문제 . . . 52
연습문제 정답 및 해설 . . . 53

5장 메모장 웹 앱 54

1. 지금까지 만든 것 합치기 . . . 54
2. 서버 — memo_app.py . . . 54
3. 화면 — memo_page.html . . . 56
4. 써 보기 . . . 58
5. 정리 . . . 59
연습문제 . . . 59
연습문제 정답 및 해설 . . . 60

6장 AI 맛보기 61

1. AI 모델 = 숫자를 넣으면 답이 나오는 함수 . . . 61
2. AI 함수 써 보기 — 서버 없이 . . . 61
3. 예측 API 와 예측 화면 만들기 . . . 63
4. 정리 . . . 66
연습문제 . . . 66
연습문제 정답 및 해설 . . . 67

7장 미니 프로젝트 69

1. 미니 프로젝트 진행 순서 . . . 69
2. 예시 프로젝트 — 우리 반 방명록 . . . 70
3. 친구 폰에서 내 서버에 접속하기 . . . 73
4. 내 프로젝트 만들기 — 템플릿에서 시작 . . . 74
5. 발표와 다음 단계 . . . 76

부록 77

부록 A. 용어 사전 77
부록 B. 명령어와 코드 모음 77
부록 C. 오류 해결 모음 78
부록 D. 스스로 더 공부하기 79

00장

웹과 API 첫걸음

웹이 움직이는 원리를 알고, 실습 환경을 준비해 첫 서버를 켜 봐요

항목	내용
예제 폴더	`fastapi/초급과정/00_웹과_API_첫걸음`
예제 파일	`ex01_json_basics.py`, `ex02_python_warmup.py`, `check_setup.py`, `ex03_first_server.py`
예상 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

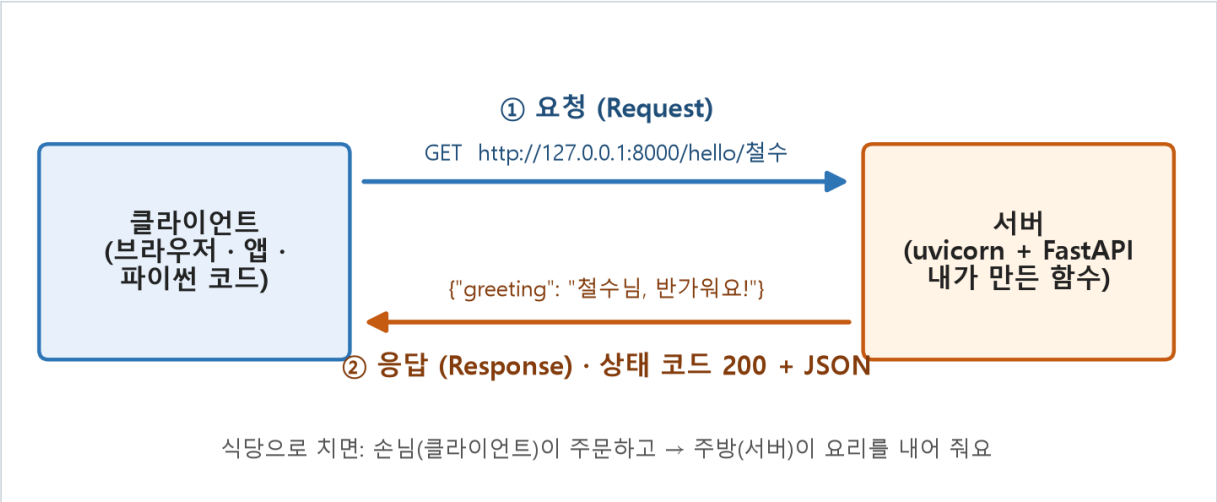
- 클라이언트 · 서버 · 요청 · 응답 · API 를 그림으로 설명할 수 있어요.
- URL 을 호스트 · 포트 · 경로 · 쿼리로 나눠 읽을 수 있어요.
- JSON 과 파이썬 딕셔너리를 서로 바꿀 수 있어요.
- 가상환경을 만들고 `fastapi`, `uvicorn` 을 설치할 수 있어요.
- `uvicorn` 으로 서버를 켜고, 브라우저로 접속하고, `Ctrl+C` 로 끌 수 있어요.

도움말 | 이번 장의 구성

- 1. 웹은 어떻게 움직일까?
- 2. URL(주소) 읽는 법
- 3. HTTP 메서드와 상태 코드
- 4. JSON 과 파이썬 딕셔너리
- 5. FastAPI 코드를 읽기 위한 파이썬 문법
- 6. 실습 환경 준비 — 폴더 · 가상환경 · 설치
- 7. 첫 서버 켜고 브라우저로 접속하기
- 8. 자주 막히는 곳과 정리
- 연습문제 · 정답 및 해설

1. 웹은 어떻게 움직일까?

브라우저 주소창에 `naver.com` 을 입력하고 Enter 를 누르면 무슨 일이 일어날까요? 내 컴퓨터의 브라우저가 멀리 있는 **서버** 컴퓨터에 "첫 화면 주세요" 라고 **요청(request)** 을 보내고, 서버는 화면에 쓸 내용을 **응답(response)** 으로 돌려줘요. 웹의 모든 일은 이 **요청 → 응답** 한 번의 반복이에요.



요청과 응답 — 클라이언트가 요청하고 서버가 응답해요

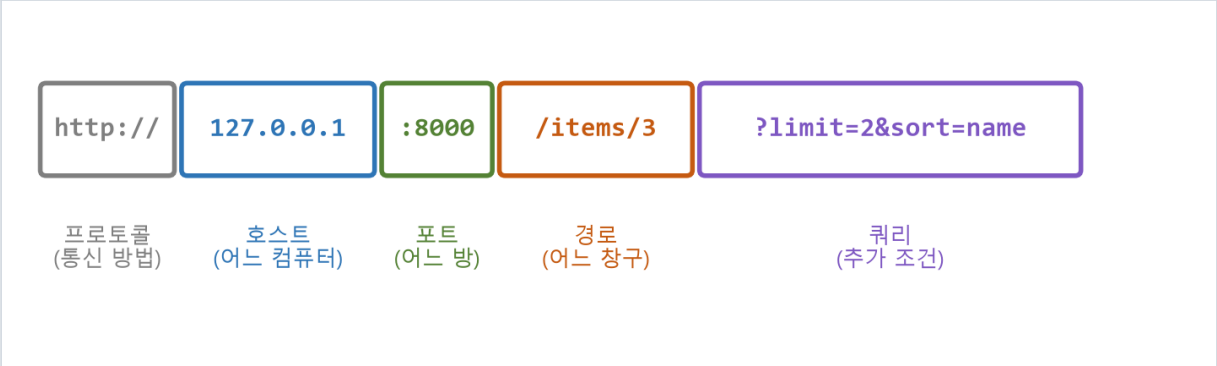
말	뜻	이 과정에서는
클라이언트	요청을 보내는 쪽	브라우저, `/docs` 화면, 웹 페이지
서버	요청을 받아 처리하고 응답하는 프로그램	`uvicorn` 으로 켜는 FastAPI 앱
API	프로그램끼리 정해진 방식으로 데이터를 주고받는 창구	우리가 만드는 주소(`/hello`, `/predict` …)
JSON	데이터를 주고받는 글자 형식	`{"name": "철수", "age": 20}`

비유로 이해하기 | 웹 페이지와 웹 API 는 무엇이 다를까?

웹 페이지는 사람이 보는 **화면(HTML)** 을 돌려주고, 웹 API 는 프로그램이 쓰기 좋은 **데이터(JSON)** 를 돌려줘요. 날씨 앱은 날씨 API 에서 JSON 을 받아 예쁜 화면으로 그려 줘요. 이 과정에서는 1~3장에서 **API(JSON 을 돌려주는 서버)** 를 만들고, 4~5장에서 그 API 를 쓰는 **웹 페이지**를 붙여서, 7장에서 나만의 웹 서비스를 완성해요.

2. URL(주소) 읽는 법

API 를 쓸 때는 주소를 정확히 읽을 줄 알아야 해요. 주소 하나는 다섯 부분으로 나뉘어요.



URL 은 프로토콜 · 호스트 · 포트 · 경로 · 쿼리로 나뉘어요

부분	예	뜻
호스트	`127.0.0.1` (= `localhost`)	**내 컴퓨터 자신**을 가리키는 특별한 주소. 실습 서버는 모두 내 컴퓨터에서 돌아요
포트	`:8000`	한 컴퓨터 안의 "방 번호". 서버 프로그램마다 다른 방을 써요 (`uvicorn` 기본값 8000)
경로	`/items/3`	어느 창구(API)에 요청하는지. FastAPI 의 `@app.get("/items/{item_id}")` 와 짝이 맞아요

부분	예	뜻
쿼리	`?limit=2&sort=name`	추가 조건. `?` 뒤에 `이름=값` 을 `&` 로 이어 붙여요

> **도움말 | 한글 주소가 이상한 글자로 바뀌어요**
> 주소에 한글을 넣으면 `/hello/%ED%99%8D...` 처럼 바뀌어 보낼 때가 있어요. 주소에는 영문·숫자만 안전하게 쓸 수 있어서 한글을 이렇게 바꿔(인코딩) 보내는 것이고, 서버는 다시 한글로 되돌려 받아요. 고장이 아니에요.

3. HTTP 메서드와 상태 코드

요청에는 "무엇을 하고 싶은지" 를 나타내는 **메서드** 가 붙고, 응답에는 "어떻게 됐는지" 를 나타내는 **상태 코드** 숫자가 붙어요. 지금은 아래 표만 눈에 익혀 두면 돼요. 1~3장에서 하나씩 직접 써 봐요.

메서드	뜻	예
GET	가져오기(조회)	글 목록 보기 — 브라우저 주소창에 입력하면 항상 GET
POST	보내서 만들기 · 처리 요청	회원 가입, 예측 요청
PUT / PATCH	고치기 (전체 / 일부)	내 정보 수정
DELETE	지우기	글 삭제

상태 코드	뜻	언제 보게 되나요?
200 OK	성공	정상 응답
201 Created	만들기 성공	새 데이터를 만들었을 때 (3장)
404 Not Found	그런 주소·데이터가 없어요	주소 오타, 없는 번호 조회
422 Unprocessable	보낸 값의 형식이 틀렸어요	숫자 자리에 글자를 보냈을 때 (1~2장)
500 Internal Error	서버 코드에서 오류가 났어요	내 함수 안의 버그 → 서버 터미널의 오류 메시지 확인

> **알아 두기 | 첫 자리만 보면 돼요**
> **2xx** 는 성공, **4xx** 는 "요청한 쪽(클라이언트)이 뭔가 잘못 보냈어요", **5xx** 는 "서버 쪽 코드에 문제가 있어요" 예요. 오류가 나면 첫 자리부터 보고 어디를 고칠지 정해요.

4. JSON 과 파이썬 딕셔너리

서버와 클라이언트는 네트워크로 **글자(문자열)** 만 주고받을 수 있어요. 그래서 파이썬 딕셔너리를 **JSON 이라는 글자 형식**으로 바꿔 보내고, 받은 쪽은 다시 딕셔너리로 바꿔 써요. FastAPI 는 이 변환을 자동으로 해 주지만, 무슨 일이 일어나는지 한 번 직접 해 봐요.

예제 파일 `fastapi/초급과정/00_웹과_API_첫걸음/ex01_json_basics.py`

ex01_json_basics.py

```python
# JSON 과 파이썬 딕셔너리 — 웹에서 데이터를 주고받는 글자 형식 익히기 (python 으로 실행)
# 서버는 파이썬 딕셔너리를 JSON 글자로 바꿔 보내고, 받는 쪽은 그 글자를 다시 딕셔너리로 바꿔요.
import json

# 1) 파이썬 딕셔너리 (내 프로그램 안에서 쓰는 값)
student = {"name": "홍길동", "age": 20, "scores": [90, 85], "is_member": True, "memo": None}
print("딕셔너리:", student)
print("타입:", type(student))

# 2) 딕셔너리 → JSON 글자 (서버가 응답을 보낼 때 하는 일)
#    ensure_ascii=False: 한글을 홍 같은 기호 대신 글자 그대로
text = json.dumps(student, ensure_ascii=False)
```

ex01_json_basics.py

```
print("\nJSON 글자:", text)
print("타입:", type(text))                    # str — 네트워크로는 글자만 보낼 수 있어요

# 파이썬과 JSON 표기 차이: True → true, None → null, 작은따옴표 → 큰따옴표

# 3) JSON 글자 → 딕셔너리 (응답을 받은 쪽이 하는 일)
back = json.loads(text)
print("\n다시 딕셔너리:", back)
print("이름만 꺼내기:", back["name"])
print("첫 번째 점수:", back["scores"][0])

# 4) 보기 좋게 출력하기 (indent=2: 두 칸 들여쓰기) — 이 과정의 _client.py 파일들이 쓰는 방법
print("\n보기 좋게:")
print(json.dumps(student, ensure_ascii=False, indent=2))
```

▶ 실행 결과

```
딕셔너리: {'name': '홍길동', 'age': 20, 'scores': [90, 85], 'is_member': True, 'memo': None}
타입: <class 'dict'>

JSON 글자: {"name": "홍길동", "age": 20, "scores": [90, 85], "is_member": true, "memo": null}
타입: <class 'str'>

다시 딕셔너리: {'name': '홍길동', 'age': 20, 'scores': [90, 85], 'is_member': True, 'memo': None}
이름만 꺼내기: 홍길동
첫 번째 점수: 90

보기 좋게:
{
  "name": "홍길동",
  "age": 20,
  "scores": [
    90,
    85
  ],
  "is_member": true,
  "memo": null
}
```

파이썬	JSON	주의
`True` / `False`	`true` / `false`	소문자
`None`	`null`	
`'글자'` 또는 `"글자"`	`"글자"`	JSON 은 큰따옴표만
`{"키": 값}`	`{"키": 값}`	키는 항상 글자

5. FastAPI 코드를 읽기 위한 파이썬 문법

FastAPI 예제에는 **타입 힌트, 기본값,** `str | None`**, 데코레이터(@), import** 다섯 가지가 계속 나와요. 처음 보는 문법이 있다면 아래 예제를 실행하며 익혀 두세요. 특히 **데코레이터**는 "함수 위에 붙이는 등록 스티커" 라고만 이해해도 충분해요.

예제 파일 `fastapi/초급과정/00_웹과_API_첫걸음/ex02_python_warmup.py`

ex02_python_warmup.py

```python
# FastAPI 코드를 읽기 위한 파이썬 문법 미리보기 (python 으로 실행)
# FastAPI 예제에는 아래 다섯 가지가 계속 나와요. 여기서 먼저 익숙해져요.

# ── 1) 타입 힌트: "이 변수에는 이런 값이 들어와요" 라는 표시 ──
# FastAPI 는 이 표시를 보고 들어온 값을 자동으로 바꾸고(변환) 검사해요.
def add(a: int, b: int) -> int:                # a, b 는 정수, 돌려주는 값도 정수
    return a + b

print("1) add(3, 4) =", add(3, 4))

# ── 2) 기본값이 있는 매개변수: 안 보내면 기본값을 써요 ──
# FastAPI 에서는 "기본값이 있으면 선택, 없으면 필수" 쿼리가 돼요.
def greet(name: str, lang: str = "ko") -> str:
    if lang == "en":
        return f"Hello, {name}!"
    return f"{name}님, 안녕하세요!"            # f-문자열: {} 안의 변수 값이 들어가요

print("2)", greet("철수"), "/", greet("Tom", lang="en"))

# ── 3) None 이 될 수 있는 값: str | None ──
# "글자이거나, 아예 없을 수도 있어요" 라는 뜻 (Python 3.10 이상 문법)
def search(keyword: str | None = None) -> str:
    if keyword is None:
        return "전체 목록"
    return f"'{keyword}' 검색 결과"

print("3)", search(), "/", search("마우스"))

# ── 4) 데코레이터 @: 함수 위에 붙이는 "등록 스티커" ──
# FastAPI 에서는 이렇게 써요:
#     @app.get("/hello")
#     def hello():
#         return {"message": "안녕하세요"}
# 뜻: "주소 /hello 로 요청이 오면 아래 hello 함수를 실행해 줘" 라고 서버의 장부에 등록하기.
# 그 장부를 딕셔너리로 직접 만들면 아래와 같아요. @ 한 줄이 ROUTES[...] = 함수 를 대신 써 주는 거예요.
def hello():
    return {"message": "안녕하세요"}


ROUTES = {"/hello": hello}                     # 주소 → 함수 장부 (@app.get 이 대신 해 주는 일)
print("4) '/hello' 로 요청이 왔다고 치면 →", ROUTES["/hello"]())   # 장부에서 함수를 찾아 실행

# ── 5) import: 다른 파일(모듈)의 것을 가져와 쓰기 ──
# from fastapi import FastAPI  ← "fastapi 라는 설치된 묶음에서 FastAPI 를 가져와요"
from datetime import date
print("5) date 를 가져와서 쓰기:", date(2026, 3, 2).strftime("%Y년 %m월 %d일"))   # 설치 없이 쓰는 표준 모듈
```

▶ 실행 결과

```
1) add(3, 4) = 7
2) 철수님, 안녕하세요! / Hello, Tom!
3) 전체 목록 / '마우스' 검색 결과
4) '/hello' 로 요청이 왔다고 치면 → {'message': '안녕하세요'}
5) date 를 가져와서 쓰기: 2026년 03월 02일
```

> **도움말 | 데코레이터는 "등록 한 줄" 이면 충분해요**
> `@app.get("/주소")` 를 함수 위에 붙이면 FastAPI 가 "그 주소로 요청이 오면 이 함수를 실행" 하도록 장부에 적어 둬요. 예제 4) 처럼 장부(딕셔너리)에 직접 적는 한 줄을 `@` 가 대신 써 주는 거예요. 데코레이터를 직접 만드는 방법은 몰라도 FastAPI 를 쓰는 데 문제없어요.

6. 실습 환경 준비 — 폴더 · 가상환경 · 설치

① 파이썬과 편집기 확인

파이썬 3.10 이상(권장 3.12)과 VS Code 를 설치해요. VS Code 에서 **[파일] → [폴더 열기]** 로 저장소의 `fastapi/초급과정` 폴더를 열고, **[터미널] → [새 터미널]** 을 누르면 아래쪽에 명령을 입력하는 창(터미널)이 열려요.

터미널: 파이썬 버전 확인

```
python --version
```

② 가상환경 만들기와 켜기

가상환경은 **이 과정 전용 패키지 상자**예요. 다른 과목의 패키지와 섞이지 않게 해 줘요. 처음 한 번만 만들고, 터미널을 새로 열 때마다 **켜기(activate)** 만 하면 돼요. 켜지면 줄 맨 앞에 `(.venv)` 가 보여요.

터미널: 가상환경 만들기 (처음 한 번)

```
python -m venv .venv
```

운영체제	켜는 명령 (터미널을 열 때마다)
Windows (PowerShell)	`.venv\Scripts\Activate.ps1`
Windows (명령 프롬프트)	`.venv\Scripts\activate.bat`
macOS / Linux	`source .venv/bin/activate`
끄기 (공통)	`deactivate`

> **주의 | PowerShell 에서 "스크립트를 실행할 수 없으므로..." 오류가 나면**
> Windows 의 보안 설정 때문이에요. 한 번만 `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` 를 입력하고 `Y` 를 누른 뒤 다시 켜 보세요. 또는 VS Code 터미널 오른쪽 위의 ∨ 를 눌러 **Command Prompt** 로 바꾸고 `activate.bat` 을 써도 돼요.

③ 패키지 설치와 점검

터미널: 패키지 설치 (가상환경을 켠 상태에서) — 초급 과정은 이 두 개면 충분해요

```
pip install fastapi uvicorn
```

설치가 끝나면 점검 프로그램으로 확인해요. `[X]` 가 있으면 안내대로 해결하고 다시 실행해요.

예제 파일 `fastapi/초급과정/00_웹과_API_첫걸음/check_setup.py`

▶ 터미널: python check_setup.py

```
[OK]  파이썬 3.12.0
[OK]  가상환경 사용 중
[OK]  fastapi   0.162.0    (API 만들기)
[OK]  uvicorn   0.54.0     (서버 실행)
[OK]  8000번 포트 비어 있음

준비 완료! 1장으로 가세요.
```

터미널 명령	뜻
`cd 폴더이름`	그 폴더로 들어가기 (예제는 반드시 **그 예제가 있는 폴더**에서 실행해요)
`cd ..`	한 단계 위 폴더로
`dir` (Windows) / `ls` (macOS)	지금 폴더의 파일 목록 보기
`python 파일.py`	파이썬 파일 실행 (서버가 아닌 보통 파이썬 파일)
`uvicorn 파일:app --reload`	서버 켜기 (`app.py` 가 아니라 `파일이름:app`)
`Ctrl + C`	실행 중인 서버 끄기
↑ 화살표	방금 입력한 명령 다시 불러오기

7. 첫 서버 켜고 브라우저로 접속하기

이제 진짜 웹 서버를 켜 볼 거예요. 코드는 1장에서 한 줄씩 자세히 배우니, 지금은 **켜고 → 접속하고 → 끄는** 흐름만 익히세요.

예제 파일 `fastapi/초급과정/00_웹과_API_첫걸음/ex03_first_server.py`

ex03_first_server.py

```
# 가장 작은 웹 서버 – 브라우저로 접속해 보기 (서버로 실행)
# 실행: 이 폴더에서  uvicorn ex03_first_server:app --reload
# 확인: 브라우저 주소창에  http://127.0.0.1:8000  과  http://127.0.0.1:8000/time  입력
# 끄기: 서버를 켠 터미널에서 Ctrl + C
from datetime import datetime

from fastapi import FastAPI

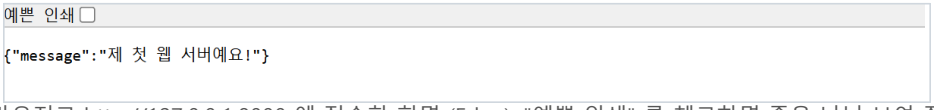
app = FastAPI()                                  # 서버(앱) 하나 만들기


@app.get("/")                                    # 주소 "/" 로 들어오면
def home():
    return {"message": "제 첫 웹 서버예요!"}     # 이 딕셔너리가 JSON 으로 브라우저에 보여요

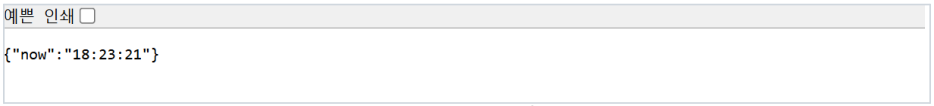

@app.get("/time")                                # 주소 "/time" 으로 들어오면
def now():
    # 요청이 올 때마다 함수가 다시 실행되므로 새로고침할 때마다 시각이 바뀌어요
    return {"now": datetime.now().strftime("%H:%M:%S")}
```

직접 해 보기 | 터미널 두 개로 따라 하기

- **터미널 1** 에서 `cd 00_웹과_API_첫걸음` → `uvicorn ex03_first_server:app --reload` 를 입력해요. `Uvicorn running on http://127.0.0.1:8000` 이 보이면 서버가 켜진 거예요. 이 터미널은 서버가 쓰고 있으니 **그대로 두세요.**
- 브라우저 주소창에 `http://127.0.0.1:8000` 을 입력해요. 아래 그림처럼 JSON 이 보이면 성공이에요.
- 주소를 `http://127.0.0.1:8000/time` 으로 바꿔 보고, 새로고침(F5)을 여러 번 눌러 시각이 바뀌는지 확인해요. 요청이 올 때마다 함수가 다시 실행되기 때문이에요.
- 터미널 1 을 보면 요청이 올 때마다 `"GET / HTTP/1.1" 200 OK` 같은 줄이 찍혀요. 서버가 받은 요청 기록이에요.
- 코드의 메시지를 바꾸고 저장(Ctrl+S)한 뒤 새로고침해 보세요. `--reload` 덕분에 서버가 저절로 다시 켜져서 바뀐 내용이 보여요.
- 다 했으면 터미널 1 에서 `Ctrl + C` 로 서버를 꺼요. 다른 명령을 입력하고 싶으면 VS Code 터미널 오른쪽 위의 **+** 로 **터미널 2** 를 열어요.



브라우저로 http://127.0.0.1:8000 에 접속한 화면 (Edge). "예쁜 인쇄" 를 체크하면 줄을 나눠 보여 줘요



http://127.0.0.1:8000/time — 새로고침할 때마다 시각이 바뀌어요

8. 자주 막히는 곳과 정리

증상	원인	해결
`'python' 은(는) 내부 또는 외부 명령…이 아닙니다`	파이썬이 설치되지 않았거나 PATH 에 없음	설치할 때 **Add python.exe to PATH** 체크, 또는 `py` 명령 사용
`No module named 'fastapi'`	가상환경이 꺼진 터미널에서 실행	줄 앞에 `(.venv)` 가 있는지 확인 → 켜고 다시 실행
`uvicorn` 을 찾을 수 없음	설치 안 됨 / 가상환경 꺼짐	`pip install uvicorn` 또는 `python -m uvicorn 파일:app`
`Could not import module "ex03_first_server"`	다른 폴더에서 서버를 켬	`cd` 로 예제 폴더에 들어가서 실행
`address already in use` / `10048`	8000번 방에 이미 다른 서버가 있음	켜 둔 서버를 `Ctrl+C` 로 끄기, 또는 `--port 8001`
브라우저에 "사이트에 연결할 수 없음"	서버가 꺼져 있음	터미널 1 에 `Uvicorn running…` 이 보이는지 확인

개념	한 줄 정리
요청 · 응답	클라이언트가 요청하고, 서버가 응답한다
URL	호스트 + 포트 + 경로 + 쿼리
메서드	GET 가져오기, POST 보내기, PUT/PATCH 고치기, DELETE 지우기
상태 코드	2xx 성공, 4xx 요청 문제, 5xx 서버 문제
JSON	딕셔너리 ↔ 글자. `json.dumps` / `json.loads`
서버 켜기	예제 폴더에서 `uvicorn 파일:app --reload`, 끄기는 `Ctrl+C`

연습문제

먼저 스스로 풀어 보고, 막히면 다음 쪽의 정답을 확인하세요.

문제 1. JSON 글자 `'{"city": "서울", "temp": 21.5, "rain": false}'` 를 딕셔너리로 바꾼 뒤 `서울의 기온은 21.5도, 비: 아니요` 처럼 출력하세요. (rain 이 true 면 "예")

문제 2. `ex03_first_server.py` 를 참고해 `GET /about` 에 `{"name": 내 이름, "course": "FastAPI"}` 를 돌려주는 서버를 만들고 브라우저로 확인하세요.

문제 3. 주소 `http://127.0.0.1:8080/books/12?lang=ko` 에서 호스트 · 포트 · 경로 · 쿼리를 각각 적어 보세요.

연습문제 정답 및 해설

문제 1 정답

문제1.py

```python
# 문제 1. JSON 글자 '{"city": "서울", "temp": 21.5, "rain": false}' 를 딕셔너리로 바꾼 뒤,
#         "서울의 기온은 21.5도, 비: 아니요" 처럼 출력하세요. (rain 이 true 면 "예")
import json

text = '{"city": "서울", "temp": 21.5, "rain": false}'
weather = json.loads(text)                    # JSON 글자 → 딕셔너리 (false → False)
rain = "예" if weather["rain"] else "아니요"
print(f"{weather['city']}의 기온은 {weather['temp']}도, 비: {rain}")
```

▶ 실행 결과

```
서울의 기온은 21.5도, 비: 아니요
```

`json.loads` 가 JSON 의 `false` 를 파이썬의 `False` 로 바꿔 줘요.

문제 2 정답

문제2.py

```python
# 문제 2. ex03_first_server.py 를 참고해 GET /about 에 {"name": 내 이름, "course": "FastAPI"} 를
#         돌려주는 서버를 만들고, 브라우저로 확인하세요.
# 실행: 정답 폴더에서  uvicorn 문제2:app --reload  →  브라우저에서 http://127.0.0.1:8000/about
from fastapi import FastAPI

app = FastAPI()


@app.get("/about")
def about():
    return {"name": "홍길동", "course": "FastAPI"}
```

▶ GET /about → 200

```
{
  "name": "홍길동",
  "course": "FastAPI"
}
```

정답 폴더에서 `uvicorn 문제2:app --reload` 로 켜고 브라우저에서 `http://127.0.0.1:8000/about` 에 접속하면 위와 같은 JSON 이 보여요.

문제 3 정답

부분	값
호스트	`127.0.0.1` (내 컴퓨터)
포트	`8080`
경로	`/books/12`
쿼리	`lang=ko` (이름 `lang`, 값 `ko`)

01장

첫 API 만들기

주소(창구)를 여러 개 만들고, 주소에 값을 실어 보내요

항목	내용
예제 폴더	`fastapi/초급과정/01_첫_API_만들기`
예제 파일	`ex01_hello.py`, `ex02_path.py`, `ex03_query.py`, (선택) `ex04_testclient.py`
예상 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- `@app.get("/주소")` 로 주소를 여러 개 만들 수 있어요.
- 경로 매개변수(`/menu/3`)와 쿼리 매개변수(`?a=3&b=5`)로 값을 받을 수 있어요.
- 422 오류를 읽고 무엇이 틀렸는지 알 수 있어요.
- `/docs` 화면에서 API 를 시험할 수 있어요.

도움말 | 이번 장의 구성

- 1. 주소(창구)를 여러 개 만들기
- 2. 경로 매개변수 — 주소 안에 값 넣기
- 3. 쿼리 매개변수 — 주소 뒤 ? 에 값 붙이기
- 4. /docs — 자동으로 생기는 API 시험 화면
- 5. (한 번만 보고 넘어가요) 코드로 요청 보내기
- 6. 자주 막히는 곳과 정리
- 연습문제 · 정답 및 해설

1. 주소(창구)를 여러 개 만들기

0장에서 켜 본 서버는 주소가 `/` 하나뿐이었어요. 이번에는 **주소를 여러 개** 만들어 볼게요. 식당으로 치면 "주문 창구", "계산 창구", "포장 창구" 를 따로 여는 것과 같아요.

예제 파일 `fastapi/초급과정/01_첫_API_만들기/ex01_hello.py`

ex01_hello.py

```python
# 1장 예제 1: 주소(창구) 여러 개 만들기
# 실행: 이 폴더에서  uvicorn ex01_hello:app --reload
# 확인: 브라우저에서 http://127.0.0.1:8000 , /hello , /about 을 차례로 열어 보세요
from fastapi import FastAPI

app = FastAPI()                        # 서버(앱) 만들기. 이름은 보통 app 이라고 지어요


# 주소 "/" → 첫 화면
@app.get("/")
def home():
    return {"message": "환영합니다! /hello 와 /about 도 열어 보세요"}


# 주소 "/hello" → 인사
@app.get("/hello")
```

ex01_hello.py

```
def hello():
    return {"message": "안녕하세요!"}

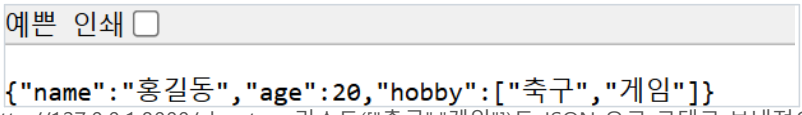

# 주소 "/about" → 자기소개 (값을 여러 개 담아도 돼요)
@app.get("/about")
def about():
    return {"name": "홍길동", "age": 20, "hobby": ["축구", "게임"]}
```

FastAPI 코드는 항상 이 **세 줄 모양**의 반복이에요.

줄	뜻
`@app.get("/hello")`	"주소 `/hello` 로 들어오는 요청은 아래 함수가 맡아요" 라고 등록 (0장 5절의 등록 스티커)
`def hello():`	그 주소를 맡을 함수. 이름은 마음대로 지어도 돼요
`return {...}`	딕셔너리를 돌려주면 FastAPI 가 **JSON 으로 바꿔** 브라우저에 보내 줘요

직접 해 보기 | 서버 켜고 주소 바꿔 가며 열어 보기

- 터미널에서 `cd 01_첫_API_만들기` 로 예제 폴더에 들어가요.
- `uvicorn ex01_hello:app --reload` 로 서버를 켜요. (`ex01_hello` = 파일 이름, `app` = 파일 안의 앱 이름)
- 브라우저에서 `http://127.0.0.1:8000` → `/hello` → `/about` 을 차례로 열어 봐요.
- 코드의 이름·취미를 내 것으로 바꾸고 저장(Ctrl+S)한 뒤 새로고침해 보세요. `--reload` 덕분에 바로 바뀌어요.
- `/없는주소` 를 열면 `{"detail":"Not Found"}` 가 나와요. 등록하지 않은 창구라서 404 예요.



http://127.0.0.1:8000/about — 리스트(["축구","게임"])도 JSON 으로 그대로 보내져요

2. 경로 매개변수 — 주소 안에 값 넣기

메뉴판에서 "3번 메뉴 주세요" 하듯이, **주소 안에 번호나 이름을 넣어** 보낼 수 있어요. `/menu/3` 의 `3` 처럼 주소의 일부가 값이 되는 것을 **경로 매개변수**라고 해요.

예제 파일 `fastapi/초급과정/01_첫_API_만들기/ex02_path.py`

ex02_path.py

```
# 1장 예제 2: 경로 매개변수 – 주소 안에 값을 넣어 보내기
# 실행: 이 폴더에서  uvicorn ex02_path:app --reload
# 확인: /menu/1 , /menu/3 , /menu/9 , /square/7 , /menu/abc 를 열어 보세요
from fastapi import FastAPI

app = FastAPI()

# 식당 메뉴판: 번호 → 메뉴 이름
MENU = {1: "김치찌개", 2: "된장찌개", 3: "비빔밥"}


# {number} 자리에 들어온 값이 함수의 number 로 들어와요
# number: int  ← "정수로 받을게요" 라는 표시. 주소의 "3"(글자)을 3(숫자)으로 바꿔 줘요
@app.get("/menu/{number}")
def menu(number: int):
    if number in MENU:
        return {"number": number, "food": MENU[number]}
    return {"number": number, "food": "없는 메뉴예요"}
```

ex02_path.py

```
19
20
21 # 숫자를 받아 제곱을 돌려주기 – /square/7 → 49
22 @app.get("/square/{n}")
23 def square(n: int):
24     return {"n": n, "square": n * n}
```

코드	뜻
`@app.get("/menu/{number}")`	주소의 `{number}` 자리에 아무 값이나 올 수 있어요
`def menu(number: int):`	그 값이 `number` 로 들어와요. `: int` 는 **정수로 받을게요** 라는 표시
`if number in MENU:`	메뉴판(딕셔너리)에 있는 번호인지 확인 — 파이썬에서 배운 그대로예요

`uvicorn ex02_path:app --reload` 로 켜고 브라우저로 열어 보면 이렇게 보여요.

▶ GET /menu/3 → 200

```
{
  "number": 3,
  "food": "비빔밥"
}
```

▶ GET /menu/9 → 200

```
{
  "number": 9,
  "food": "없는 메뉴예요"
}
```

▶ GET /square/7 → 200

```
{
  "n": 7,
  "square": 49
}
```

비유로 이해하기 | 숫자 자리에 글자를 넣으면? — 422 오류

`/menu/abc` 를 열어 보세요. `int` 라고 했는데 `abc` 는 정수가 아니니까, FastAPI 가 **함수를 실행하기도 전에** 막고 이유를 알려 줘요. 우리는 검사 코드를 한 줄도 안 썼는데 말이에요! 상태 코드 **422** 는 "보낸 값의 형식이 틀렸어요" 라는 뜻이에요.

예쁜 인쇄 ☐

{"detail":[{"type":"int_parsing","loc":["path","number"],"msg":"Input should be a valid

/menu/abc — "Input should be a valid integer" (정수를 넣어 주세요)

422 오류 읽는 법	이 예에서는
`loc` (위치)	`["path", "number"]` → 주소(path)의 number 자리가 틀렸어요
`msg` (이유)	`Input should be a valid integer` → 정수여야 해요
`input` (보낸 값)	`"abc"`

3. 쿼리 매개변수 — 주소 뒤 ? 에 값 붙이기

검색창에 글자를 넣고 검색하면 주소 끝에 `?q=검색어` 가 붙는 것을 본 적 있나요? 이렇게 `?` **뒤에** `이름=값` 을 붙여 보내는 것이 **쿼리 매개변수**예요. 값이 여러 개면 `&` 로 이어요.

예제 파일 `fastapi/초급과정/01_첫_API_만들기/ex03_query.py`

ex03_query.py

```python
# 1장 예제 3: 쿼리 매개변수 – 주소 뒤 ? 에 값을 붙여 보내기
# 실행: 이 폴더에서  uvicorn ex03_query:app --reload
# 확인: /add?a=3&b=5 , /greet?name=철수 , /greet?name=Tom&lang=en , /greet , /add?a=3 를 열어 보세요
from fastapi import FastAPI

app = FastAPI()


# 주소에 {} 가 없는 매개변수는 쿼리로 받아요:  /add?a=3&b=5
# 기본값이 없으면 "꼭 보내야 하는 값" 이에요
@app.get("/add")
def add(a: int, b: int):
    return {"a": a, "b": b, "result": a + b}


# 기본값(= "손님", = "ko")이 있으면 안 보내도 돼요
@app.get("/greet")
def greet(name: str = "손님", lang: str = "ko"):
    if lang == "en":
        return {"message": f"Hello, {name}!"}
    return {"message": f"{name}님, 안녕하세요!"}
```

코드	뜻
`def add(a: int, b: int):`	주소에 `{}` 가 없는 매개변수는 **쿼리**로 받아요 → `/add?a=3&b=5`
기본값이 없음 (`a: int`)	꼭 보내야 해요. 빠뜨리면 422
기본값이 있음 (`name: str = "손님"`)	안 보내도 돼요. 안 보내면 기본값을 써요

▶ GET /add?a=3&b=5 → 200

```
{
  "a": 3,
  "b": 5,
  "result": 8
}
```

▶ GET /greet?name=Tom&lang=en → 200

```
{
  "message": "Hello, Tom!"
}
```

▶ GET /greet → 200

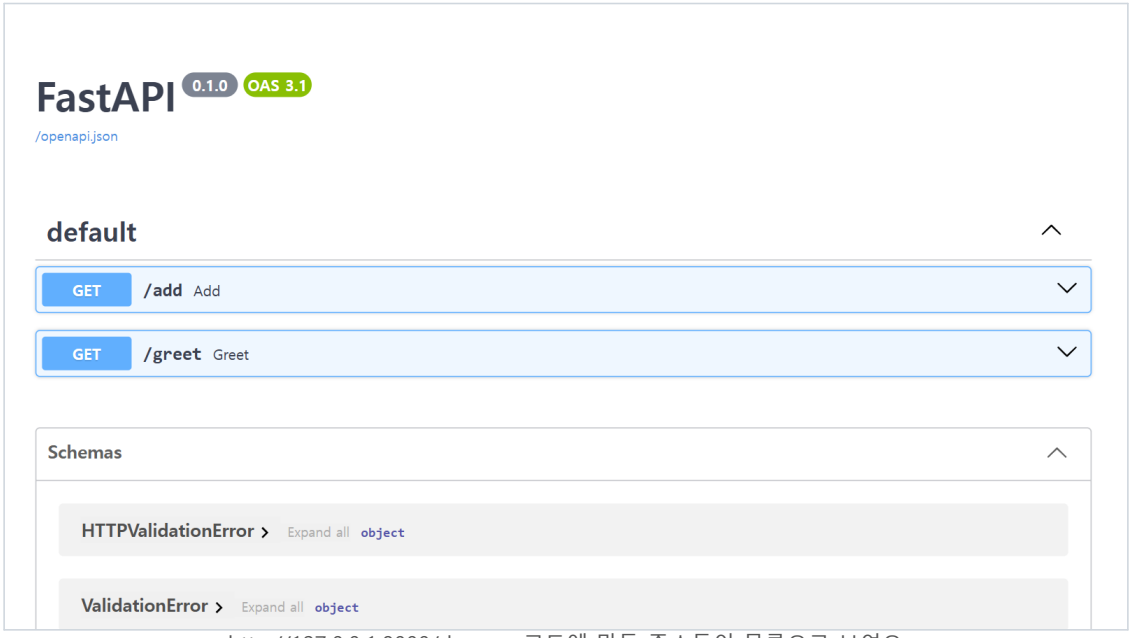
```
{
  "message": "손님님, 안녕하세요!"
}
```

도움말 | 경로와 쿼리, 언제 무엇을 쓸까?

"무엇을" 가리킬 때는 경로(`/menu/3` — 3번 메뉴), **"어떻게"** 를 덧붙일 때는 쿼리(`/greet?lang=en` — 영어로)를 쓰는 경우가 많아요. 정답이 정해진 것은 아니니 편한 쪽을 고르면 돼요.

4. /docs — 자동으로 생기는 API 시험 화면

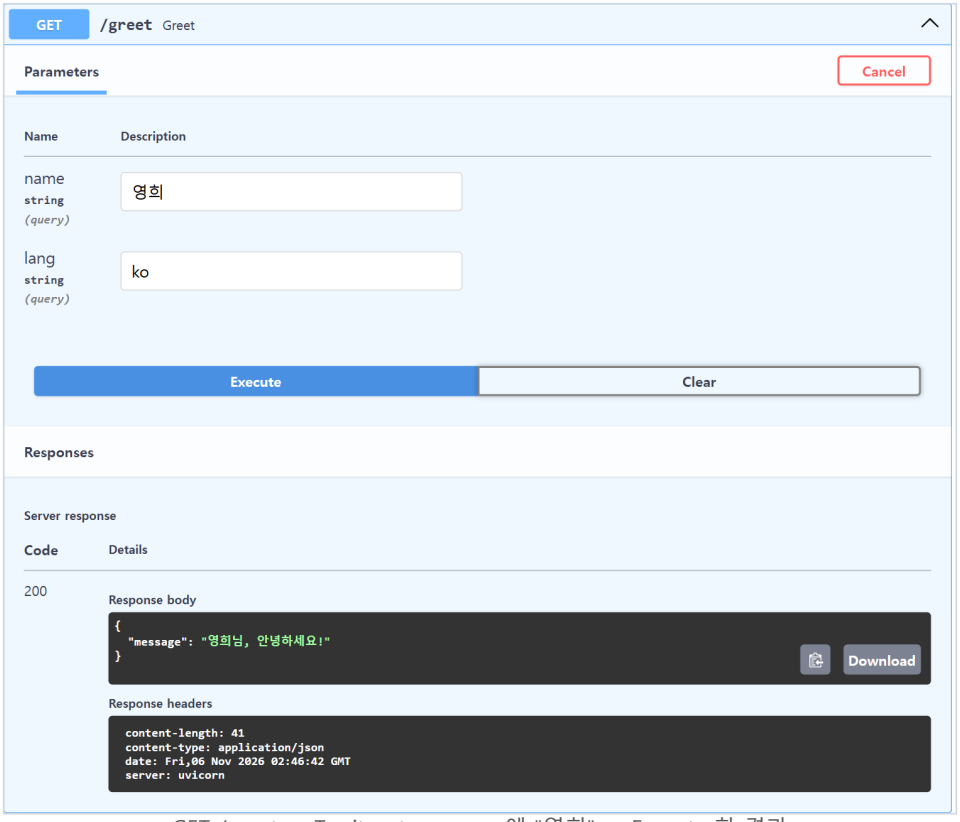
서버를 켜면 FastAPI 가 **API 설명서 겸 시험 화면**을 저절로 만들어 줘요. 주소 끝에 `/docs` 를 붙이기만 하면 돼요. 앞으로 계속 쓰게 될 아주 중요한 화면이에요.



http://127.0.0.1:8000/docs — 코드에 만든 주소들이 목록으로 보여요

직접 해 보기 | /docs 에서 시험하기 (Try it out)

- `uvicorn ex03_query:app --reload` 로 서버를 켜고 `http://127.0.0.1:8000/docs` 를 열어요.
- **GET /greet** 줄을 눌러 펼치고, 오른쪽 **[Try it out]** 버튼을 눌러요.
- `name` 칸에 이름을 적고 파란 **[Execute]** 를 눌러요.
- 아래 **Server response** 에 상태 코드(200)와 응답 JSON 이 나와요. 브라우저 주소창에 직접 쳤을 때와 같은 결과예요.



GET /greet → Try it out → name 에 "영희" → Execute 한 결과

5. (한 번만 보고 넘어가요) 코드로 요청 보내기

브라우저나 `/docs` 대신 **파이썬 코드로** 요청을 보낼 수도 있어요. 프로그램끼리 API 를 주고받는 것이 바로 이런 모습이에요. 이 교안에서는 브라우저와 `/docs` 로 확인하니, **"이런 방법도 있구나"** 만 알고 넘어가면 돼요.

예제 파일 `fastapi/초급과정/01_첫_API_만들기/ex04_testclient.py`

ex04_testclient.py

```python
# 1장 예제 4 (한 번만 보고 넘어가요): 브라우저 대신 "코드로" 요청 보내기 (python 으로 실행)
# TestClient 는 서버를 켜지 않고 앱에 요청을 보내 주는 "코드로 만든 브라우저" 예요.
# 이 과정에서는 브라우저와 /docs 로 확인하니, 이런 방법도 있다는 것만 알아 두세요.
from fastapi.testclient import TestClient

from ex02_path import app             # 예제 2의 앱 가져오기

client = TestClient(app)

res = client.get("/menu/3")           # 브라우저 주소창에 /menu/3 을 친 것과 같아요
print(res.status_code)                # 상태 코드: 200 이면 성공
print(res.json())                     # 응답 JSON → 파이썬 딕셔너리
```

▶ 터미널: python ex04_testclient.py

```
200
{'number': 3, 'food': '비빔밥'}
```

> **알아 두기 | TestClient 는 "코드로 만든 브라우저"**
>
> `client.get("/menu/3")` 한 줄 = 브라우저 주소창에 `/menu/3` 을 친 것과 같아요. 서버를 켜지 않아도 돼요. 이 예제는 `pip install httpx` 를 해야 실행돼요(선택).

6. 자주 막히는 곳과 정리

증상	원인	해결
`Could not import module "ex01_hello"`	다른 폴더에서 서버를 켬	`cd 01_첫_API_만들기` 로 예제 폴더에 들어가서 켜기
`Attribute "app" not found`	`파일:app` 의 app 이름이 코드와 다름	코드의 `app = FastAPI()` 이름 확인
고쳤는데 그대로예요	저장을 안 했거나 `--reload` 없이 켬	Ctrl+S 저장, `--reload` 붙이기
`{"detail":"Not Found"}`	주소 오타, 등록 안 한 주소	`/docs` 에서 등록된 주소 확인
422	숫자 자리에 글자, 꼭 보낼 값을 빠뜨림	응답의 `loc·msg` 읽기

배운 것	한 줄 정리
주소 만들기	`@app.get("/주소")` + 함수 + `return {...}`
경로 매개변수	`/menu/{number}` + `def menu(number: int)` → `/menu/3`
쿼리 매개변수	`def add(a: int, b: int)` → `/add?a=3&b=5`, 기본값이 있으면 생략 가능
자동 검사	타입(`int`)과 다른 값 → 422
시험 화면	`/docs` → Try it out → Execute

연습문제

문제 1. 주소 `/gugudan/{dan}` 으로 단을 받아 그 단의 구구단 9줄을 리스트로 돌려주세요. 예) `/gugudan/3` → `{"dan": 3, "result": ["3 x 1 = 3", ..., "3 x 9 = 27"]}`

문제 2. 쿼리로 키(`height`, cm)와 몸무게(`weight`, kg)를 받아 BMI 를 소수 1자리로 돌려주세요. 예) `/bmi?height=175&weight=70` → `{"bmi": 22.9}` (BMI = 몸무게 ÷ (키 ÷ 100)²)

연습문제 정답 및 해설

문제 1 정답

문제1.py

```python
# 문제 1. 주소 /gugudan/{dan} 으로 단(dan)을 받아 그 단의 구구단 9줄을 리스트로 돌려주세요.
#         예) /gugudan/3 → {"dan": 3, "result": ["3 x 1 = 3", ..., "3 x 9 = 27"]}
# 실행: 이 폴더에서  uvicorn 문제1:app --reload  →  브라우저에서 /gugudan/3
from fastapi import FastAPI

app = FastAPI()


@app.get("/gugudan/{dan}")
def gugudan(dan: int):
    lines = [f"{dan} x {i} = {dan * i}" for i in range(1, 10)]   # 1 ~ 9 를 곱한 글자 9개
    return {"dan": dan, "result": lines}
```

▶ GET /gugudan/3 → 200


```
{
  "dan": 3,
  "result": [
    "3 x 1 = 3",
    "3 x 2 = 6",
    "3 x 3 = 9",
    "3 x 4 = 12",
    "3 x 5 = 15",
    "3 x 6 = 18",
    "3 x 7 = 21",
    "3 x 8 = 24",
    "3 x 9 = 27"
  ]
}
```


`[... for i in range(1, 10)]` 는 1~9 를 차례로 넣어 글자 9개짜리 리스트를 만드는 파이썬 문법(리스트 컴프리헨션)이에요. `for` 문으로 하나씩 `append` 해도 똑같아요.

문제 2 정답

문제2.py

```python
# 문제 2. 쿼리로 키(height, cm)와 몸무게(weight, kg)를 받아 BMI 를 소수 1자리로 돌려주세요.
#         예) /bmi?height=175&weight=70 → {"bmi": 22.9}
#         BMI = 몸무게 ÷ (키 ÷ 100)²
# 실행: 이 폴더에서  uvicorn 문제2:app --reload  →  브라우저에서 /bmi?height=175&weight=70
from fastapi import FastAPI

app = FastAPI()


@app.get("/bmi")
def bmi(height: float, weight: float):       # float: 소수점이 있는 숫자도 받기
    value = weight / (height / 100) ** 2
    return {"height": height, "weight": weight, "bmi": round(value, 1)}
```

▶ GET /bmi?height=175&weight=70 → 200


```
{
  "height": 175.0,
  "weight": 70.0,
  "bmi": 22.9
}
```

```
▶ GET /bmi?height=175&weight=70 → 200
}
```

키에 `175.5` 처럼 소수점이 있는 값도 받으려고 `float` 를 썼어요. `int` 로 쓰면 `175.5` 는 422 가 돼요.

02장

데이터 보내기 — POST

회원 가입처럼 여러 값을 본문(JSON)에 담아 보내고, 서버가 검사하게 해요

항목	내용
예제 폴더	`fastapi/초급과정/02_데이터_보내기`
예제 파일	`ex01_post.py`, `ex02_field.py`, `ex03_coffee.py`
예상 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- GET 과 POST 의 차이를 설명할 수 있어요.
- `BaseModel` 로 받을 데이터의 모양을 정할 수 있어요.
- `Field` 로 값의 범위와 글자 수 규칙을 정할 수 있어요.
- `/docs` 의 Try it out 으로 POST 요청을 보낼 수 있어요.

도움말 | 이번 장의 구성

- 1. GET 과 POST — 보내는 방법이 달라요
- 2. POST 로 데이터 받기 — 회원 가입
- 3. Field 로 규칙 정하기 — 이상한 값 막기
- 4. 받은 값으로 계산하기 — 커피 주문
- 5. 자주 막히는 곳과 정리
- 연습문제 · 정답 및 해설

1. GET 과 POST — 보내는 방법이 달라요

1장에서는 값을 **주소에 실어** 보냈어요(GET). 그런데 회원 가입처럼 보낼 값이 많거나, 비밀번호처럼 주소에 보이면 안 되는 값은 어떻게 보낼까요? 이럴 때는 **POST** 요청의 **본문(body)** 에 JSON 을 담아 보내요.

비유로 이해하기 | 엽서와 소포

GET 은 **엽서**예요. 내용(값)이 주소 옆에 다 보이고, 짧은 내용만 보낼 수 있어요. POST 는 **소포**예요. 상자(본문) 안에 여러 물건(값)을 담아 보내고, 겉에서는 내용이 보이지 않아요.

	GET	POST
값을 담는 곳	주소 (`/add?a=3&b=5`)	본문 (JSON)
주로 하는 일	가져오기 · 조회	보내기 · 만들기 · 처리 요청
브라우저 주소창으로	보낼 수 있어요	**보낼 수 없어요** → `/docs` 로 시험
FastAPI 코드	`@app.get(...)`	`@app.post(...)`

2. POST 로 데이터 받기 — 회원 가입

POST 로 받을 때는 먼저 **"이런 모양의 데이터를 받을게요"** 하고 모양을 정해요. 이 모양을 정해 주는 도구가 **Pydantic** 이고, `BaseModel` 을 물려받는 클래스로 만들어요.

예제 파일 `fastapi/초급과정/02_데이터_보내기/ex01_post.py`

ex01_post.py

```python
# 2장 예제 1: POST 로 데이터 받기 – 회원 가입
# 실행: 이 폴더에서  uvicorn ex01_post:app --reload
# 확인: POST 는 주소창으로 보낼 수 없어요 → http://127.0.0.1:8000/docs 의 Try it out 으로 시험해요
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

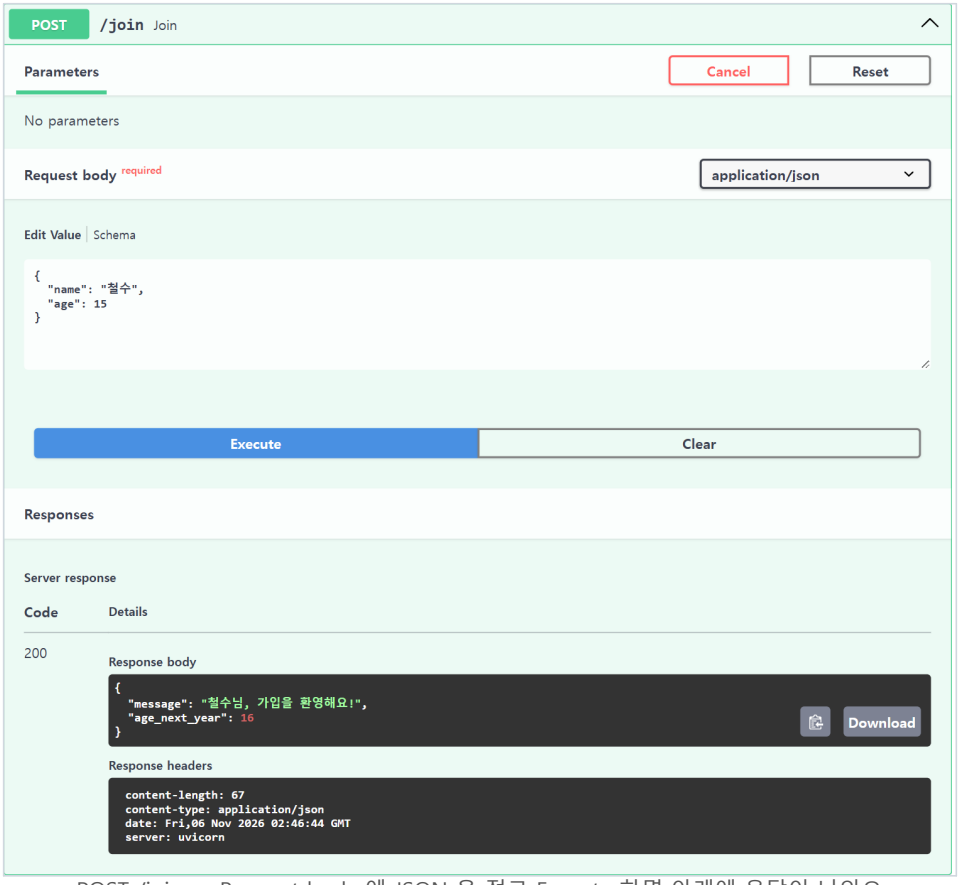

# 받을 데이터의 "모양" 정하기 – 이름은 글자, 나이는 정수
# BaseModel 을 물려받아 클래스를 만들면, 들어온 JSON 을 이 모양대로 검사해 줘요
class Member(BaseModel):
    name: str
    age: int


# @app.post: POST 요청을 받는 창구. 본문(JSON)이 member 로 들어와요
@app.post("/join")
def join(member: Member):
    return {"message": f"{member.name}님, 가입을 환영해요!", "age_next_year": member.age + 1}
```

코드	뜻
`class Member(BaseModel):`	받을 데이터의 모양(설계도). 클래스 이름은 마음대로
`name: str` / `age: int`	필드 이름과 자료형. 보낼 JSON 이 `{"name": "...", "age": 숫자}` 모양이어야 해요
`@app.post("/join")`	POST 요청을 받는 창구
`def join(member: Member):`	본문 JSON 이 `Member` 모양으로 바뀌어 `member` 에 들어와요
`member.name`	받은 값 꺼내기 (딕셔너리의 `["name"]` 대신 점(.)으로)

직접 해 보기 | /docs 에서 POST 보내기

- `uvicorn ex01_post:app --reload` 로 켜고 `http://127.0.0.1:8000/docs` 를 열어요.
- **POST /join** 을 펼치고 **[Try it out]** 을 눌러요.
- **Request body** 칸의 JSON 에서 `"string"` 과 `0` 을 내 이름과 나이로 바꿔요.
- **[Execute]** 를 누르고 아래 응답을 확인해요.



POST /join — Request body 에 JSON 을 적고 Execute 하면 아래에 응답이 나와요

같은 요청을 보냈을 때의 결과를 글자로 보면 이래요.

▶ POST /join → 200

```
{
  "message": "철수님, 가입을 환영해요!",
  "age_next_year": 16
}
```

> **주의 | 나이에 "열다섯" 을 보내면?**
> `age` 를 `int` 로 정했으니 글자를 보내면 1장처럼 **422** 가 와요. Pydantic 이 모양을 검사해 준 거예요.

3. Field 로 규칙 정하기 — 이상한 값 막기

나이에 `-5` 나 `200` 이 들어오면 어떨까요? 자료형(int)은 맞지만 말이 안 되는 값이에요. `Field` 로 **범위와 글자 수** 규칙을 정해 두면, 규칙에 어긋나는 값은 FastAPI 가 알아서 막아 줘요.

예제 파일 `fastapi/초급과정/02_데이터_보내기/ex02_field.py`

ex02_field.py

```python
# 2장 예제 2: Field 로 값의 규칙 정하기 — 이상한 값은 서버가 알아서 막아요
# 실행: 이 폴더에서  uvicorn ex02_field:app --reload
# 확인: /docs 에서 age 에 -5 나 200, name 에 "" 를 넣어 보내 보세요 → 422 오류
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Member(BaseModel):
```

ex02_field.py

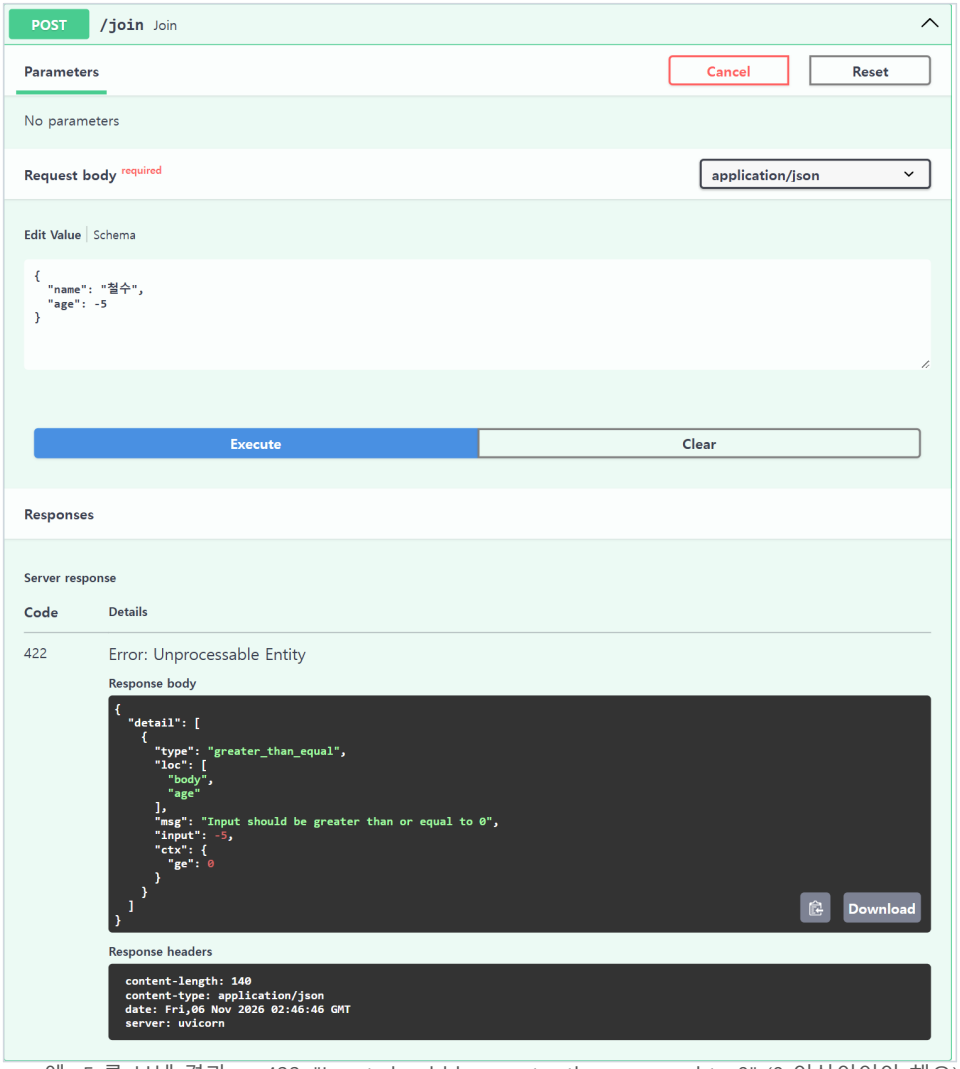
```python
    # min_length / max_length : 글자 수 범위
    name: str = Field(min_length=1, max_length=10)
    # ge(크거나 같다) / le(작거나 같다) : 숫자 범위
    age: int = Field(ge=0, le=150)


@app.post("/join")
def join(member: Member):
    # 여기까지 왔다면 규칙을 모두 통과한 값이에요 (검사 코드를 따로 쓸 필요가 없어요)
    return {"message": f"{member.name}님, 가입을 환영해요!"}
```

규칙	뜻	예
`ge=0`	0 보다 크거나 같다 (greater or equal)	나이 ≥ 0
`le=150`	150 보다 작거나 같다 (less or equal)	나이 ≤ 150
`gt=0`	0 보다 크다 (0 은 안 돼요)	가격 > 0
`min_length=1`	최소 1글자 (빈 글자 금지)	이름
`max_length=10`	최대 10글자	이름
`default=1`	안 보내면 쓸 기본값	수량

▶ POST /join → 422


```
{
  "detail": [
    {
      "type": "greater_than_equal",
      "loc": [
        "body",
        "age"
      ],
      "msg": "Input should be greater than or equal to 0",
      "input": -5,
      "ctx": {
        "ge": 0
      }
    }
  ]
}
```



age 에 -5 를 보낸 결과 — 422, "Input should be greater than or equal to 0" (0 이상이어야 해요)

> **알아 두기 | 검사 코드를 직접 안 써도 되는 이유**
>
> `if member.age < 0: ...` 같은 검사를 하나하나 쓰지 않아도, 규칙을 어긴 값은 **함수에 들어오기 전에** 막혀요. 그래서 함수 안에서는 "규칙을 통과한 좋은 값" 만 다루면 돼요.

4. 받은 값으로 계산하기 — 커피 주문

이제 받은 값으로 무언가를 계산해서 돌려줘 볼게요. 메뉴와 크기, 잔 수를 받아 가격을 계산하는 커피 주문 API 예요.

예제 파일 `fastapi/초급과정/02_데이터_보내기/ex03_coffee.py`

ex03_coffee.py

```python
# 2장 예제 3: 커피 주문 받기 – 받은 값으로 계산해서 돌려주기
# 실행: 이 폴더에서  uvicorn ex03_coffee:app --reload
# 확인: /docs 에서 POST /order 를 열고 menu, size, count 를 바꿔 가며 보내 보세요
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

PRICE = {"아메리카노": 3000, "라떼": 3500, "녹차": 3200}      # 메뉴별 가격 (M 사이즈)


```

ex03_coffee.py

```python
class Order(BaseModel):
    menu: str                                  # 메뉴 이름
    size: str = "M"                            # 기본값이 있으면 안 보내도 돼요 (S, M, L)
    count: int = Field(default=1, ge=1, le=10) # 1 ~ 10잔


@app.post("/order")
def order(o: Order):
    if o.menu not in PRICE:
        return {"error": f"{o.menu} 는 없는 메뉴예요", "menus": list(PRICE)}
    price = PRICE[o.menu]
    if o.size == "L":                          # L 사이즈는 500원 더
        price += 500
    return {"menu": o.menu, "size": o.size, "count": o.count, "total": price * o.count}
```

▶ POST /order → 200

```
{
  "menu": "라떼",
  "size": "L",
  "count": 2,
  "total": 8000
}
```

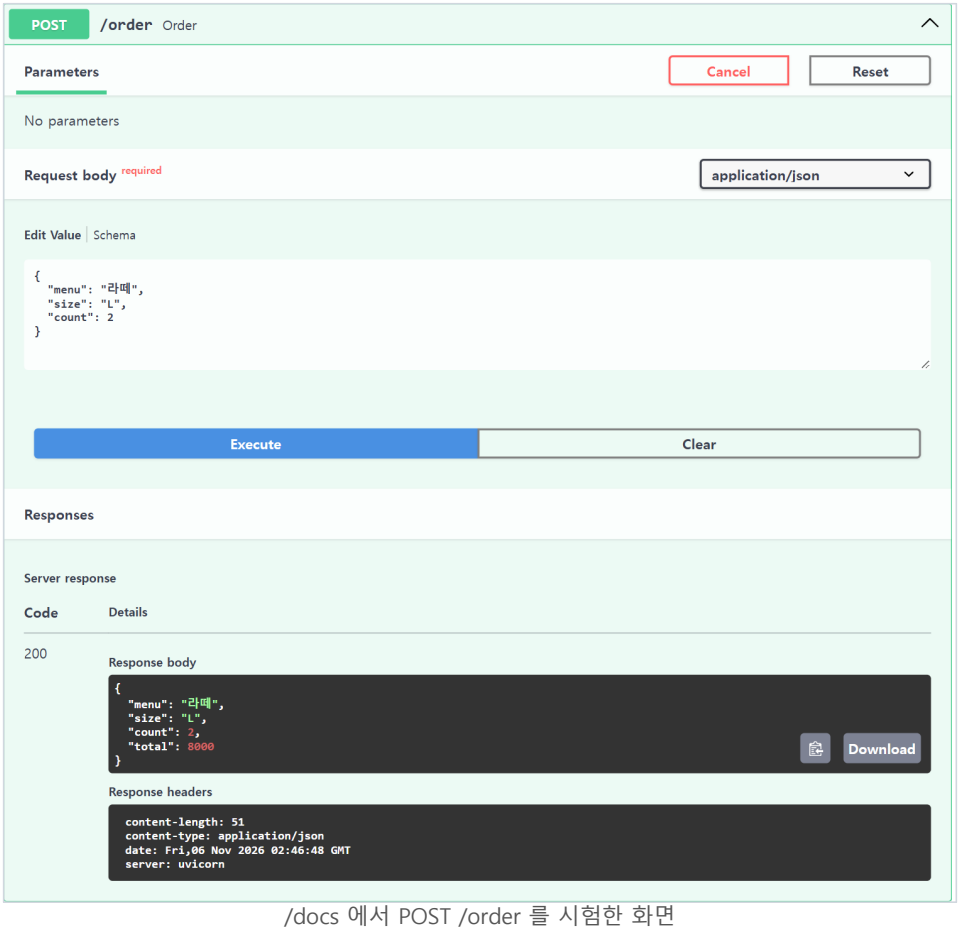
▶ POST /order → 200

```
{
  "menu": "아메리카노",
  "size": "M",
  "count": 1,
  "total": 3000
}
```

▶ POST /order → 200

```
{
  "error": "콜라 는 없는 메뉴예요",
  "menus": [
    "아메리카노",
    "라떼",
    "녹차"
  ]
}
```

두 번째 요청은 `size` 와 `count` 를 안 보냈지만 기본값(`"M"`, `1`)으로 계산됐어요. 세 번째는 메뉴판에 없는 메뉴라서 우리가 만든 안내 메시지를 돌려줬어요.



/docs 에서 POST /order 를 시험한 화면

5. 자주 막히는 곳과 정리

증상	원인	해결
브라우저 주소창에 `/join` 을 치면 `Method Not Allowed`	주소창은 항상 GET 요청	POST 는 `/docs` 의 Try it out 으로
422 + `"type": "missing"`	꼭 보낼 값(기본값 없는 필드)을 빠뜨림	`loc` 에 적힌 필드를 넣어 보내기
422 + `greater_than_equal` 등	Field 규칙을 어김	`msg` 를 읽고 값 고치기
JSON 칸에서 오류	쉼표·따옴표가 빠짐, 마지막에 쉼표	JSON 은 큰따옴표만, 마지막 항목 뒤에 쉼표 금지

배운 것	한 줄 정리
POST	본문(JSON)에 값을 담아 보내는 요청. `@app.post(...)`
받을 모양	`class 이름(BaseModel):` + `필드: 자료형`
값 꺼내기	`member.name` 처럼 점(.)으로
규칙	`Field(ge=, le=, gt=, min_length=, max_length=, default=)`
시험	`/docs` → Try it out → Request body 수정 → Execute

연습문제

문제 1. 이름과 국어·영어·수학 점수를 `POST /score` 로 받아 총점과 평균(소수 1자리)을 돌려주세요. 점수는 0 ~ 100 만 받도록 `Field` 로 규칙을 정하세요.

문제 2. 상품 이름(`name`), 가격(`price`, 0 보다 큼), 할인율(`discount`, 0 ~ 50, 기본값 0)을 받아 할인된 가격을 돌려주는 `POST /sale` 을 만드세요.

연습문제 정답 및 해설

문제 1 정답

문제1.py

```python
# 문제 1. 이름과 국어·영어·수학 점수를 POST /score 로 받아 총점과 평균(소수 1자리)을 돌려주세요.
#         점수는 0 ~ 100 만 받도록 Field 로 규칙을 정하세요.
#         예) {"name": "영희", "kor": 90, "eng": 85, "math": 70} → {"name": "영희", "total": 245, "average": 81.7}
# 실행: 이 폴더에서  uvicorn 문제1:app --reload  →  /docs 에서 시험
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Score(BaseModel):
    name: str
    kor: int = Field(ge=0, le=100)
    eng: int = Field(ge=0, le=100)
    math: int = Field(ge=0, le=100)


@app.post("/score")
def score(s: Score):
    total = s.kor + s.eng + s.math
    return {"name": s.name, "total": total, "average": round(total / 3, 1)}
```

▶ POST /score → 200

```
{
  "name": "영희",
  "total": 245,
  "average": 81.7
}
```

▶ POST /score → 422

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "kor"
      ],
      "msg": "Input should be less than or equal to 100",
      "input": 120,
      "ctx": {
        "le": 100
      }
    }
  ]
}
```

국어에 120 을 보내면 `le=100` 규칙에 걸려 422 가 와요.

문제 2 정답

문제2.py

```python
# 문제 2. 상품 이름(name)과 가격(price, 0 보다 큼), 할인율(discount, 0 ~ 50, 기본값 0)을 받아
#         할인된 가격을 돌려주는 POST /sale 을 만드세요.
#         예) {"name": "운동화", "price": 50000, "discount": 20} → {"name": "운동화", "sale_price": 40000}
# 실행: 이 폴더에서  uvicorn 문제2:app --reload  →  /docs 에서 시험
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class Item(BaseModel):
    name: str
    price: int = Field(gt=0)                       # gt: 0 보다 커야 해요 (0 은 안 돼요)
    discount: int = Field(default=0, ge=0, le=50)  # 0 ~ 50 %


@app.post("/sale")
def sale(item: Item):
    sale_price = item.price * (100 - item.discount) // 100   # // : 소수점 버린 나눗셈
    return {"name": item.name, "sale_price": sale_price}
```

▶ POST /sale → 200

```
{
  "name": "운동화",
  "sale_price": 40000
}
```

// 는 나눗셈 결과의 소수점을 버리는 연산자예요. 가격이 원 단위 정수로 나와요.

03장

메모장 API — CRUD

만들기 · 읽기 · 고치기 · 지우기를 갖춘 메모장 API 를 만들어요

항목	내용
예제 폴더	`fastapi/초급과정/03_메모장_API`
예제 파일	`ex01_memo.py`
예상 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- CRUD 와 HTTP 메서드(POST · GET · PUT · DELETE)를 짝지을 수 있어요.
- 딕셔너리로 데이터를 저장하고 꺼낼 수 있어요.
- 없는 데이터를 찾으면 `HTTPException` 으로 404 를 보낼 수 있어요.

도움말 | 이번 장의 구성

- 1. CRUD — 거의 모든 웹 서비스의 뼈대
- 2. 메모장 API 만들기
- 3. 없는 메모는 404 — HTTPException
- 4. /docs 에서 차례로 시험하기
- 5. 자주 막히는 곳과 정리
- 연습문제 · 정답 및 해설

1. CRUD — 거의 모든 웹 서비스의 뼈대

메모장, 게시판, 쇼핑몰 장바구니, 할 일 목록... 모양은 달라도 하는 일은 네 가지뿐이에요. **만들고(Create) · 읽고(Read) · 고치고(Update) · 지우기(Delete).** 앞 글자를 따서 **CRUD** 라고 해요. 그리고 HTTP 메서드가 이 네 가지와 짝이 맞아요.

CRUD	하는 일	HTTP 메서드	메모장에서는
Create	만들기	POST	새 메모 쓰기
Read	읽기	GET	메모 목록 보기, 메모 하나 보기
Update	고치기	PUT	메모 내용 고치기
Delete	지우기	DELETE	메모 지우기

이번 장에서는 1~2장에서 배운 것만으로 **메모장 API** 를 만들어요. 새로 나오는 것은 **없는 메모를 찾을 때 404 오류 보내기** 하나뿐이에요.

2. 메모장 API 만들기

메모는 파이썬 **딕셔너리**에 저장해요. `{1: "우유 사기", 2: "숙제하기"}` 처럼 **번호 → 메모 글자** 모양이에요.

예제 파일 `fastapi/초급과정/03_메모장_API/ex01_memo.py`

ex01_memo.py

```python
# 3장 예제: 메모장 API — 만들기(Create) · 읽기(Read) · 고치기(Update) · 지우기(Delete)
# 실행: 이 폴더에서  uvicorn ex01_memo:app --reload
# 확인: http://127.0.0.1:8000/docs 에서 위에서부터 차례로 Try it out 해 보세요
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="메모장 API")


# 메모를 만들거나 고칠 때 받는 모양
class MemoIn(BaseModel):
    text: str = Field(min_length=1, max_length=100)


# 메모를 저장할 곳: 번호 → 메모 글자 (서버를 끄면 사라져요. 7장에서 파일에 저장해 봐요)
memos = {}
next_id = 1                                  # 다음에 만들 메모 번호


# C: 메모 만들기 — status_code=201 은 "새로 만들었어요" 라는 성공 코드
@app.post("/memos", status_code=201)
def create_memo(memo: MemoIn):
    global next_id                           # 함수 밖의 next_id 를 바꾸겠다는 표시
    memos[next_id] = memo.text
    result = {"id": next_id, "text": memo.text}
    next_id += 1
    return result


# R: 메모 전체 보기
@app.get("/memos")
def list_memos():
    return [{"id": i, "text": t} for i, t in memos.items()]


# R: 메모 하나 보기 — 없는 번호면 404 오류
@app.get("/memos/{memo_id}")
def read_memo(memo_id: int):
    if memo_id not in memos:
        # raise HTTPException: "여기서 멈추고 이 오류를 응답으로 보내요"
        raise HTTPException(status_code=404, detail=f"{memo_id}번 메모가 없어요")
    return {"id": memo_id, "text": memos[memo_id]}


# U: 메모 고치기
@app.put("/memos/{memo_id}")
def update_memo(memo_id: int, memo: MemoIn):
    if memo_id not in memos:
        raise HTTPException(status_code=404, detail=f"{memo_id}번 메모가 없어요")
    memos[memo_id] = memo.text
    return {"id": memo_id, "text": memo.text}


# D: 메모 지우기
@app.delete("/memos/{memo_id}")
def delete_memo(memo_id: int):
    if memo_id not in memos:
        raise HTTPException(status_code=404, detail=f"{memo_id}번 메모가 없어요")
    del memos[memo_id]
    return {"message": f"{memo_id}번 메모를 지웠어요"}
```

줄	뜻
`memos = {}` / `next_id = 1`	메모를 담을 딕셔너리와 다음에 줄 번호. 서버가 켜져 있는 동안만 기억해요
`@app.post("/memos", status_code=201)`	만들기 성공은 200 대신 **201(Created)** 로 알려 주는 게 약속이에요
`global next_id`	함수 안에서 함수 밖의 변수를 **바꿀** 때 붙이는 표시 (파이썬 문법)
`@app.get("/memos/{memo_id}")`	1장의 경로 매개변수 — 몇 번 메모인지 주소로 받아요
`@app.put(...)` / `@app.delete(...)`	고치기 · 지우기 창구. 쓰는 법은 get, post 와 같아요

3. 없는 메모는 404 — HTTPException

`/memos/9` 처럼 없는 번호를 달라고 하면 어떻게 할까요? "그런 메모 없어요" 라는 **오류 응답(404)** 을 보내야 해요. 이때 쓰는 것이 `raise HTTPException(...)` 이에요.

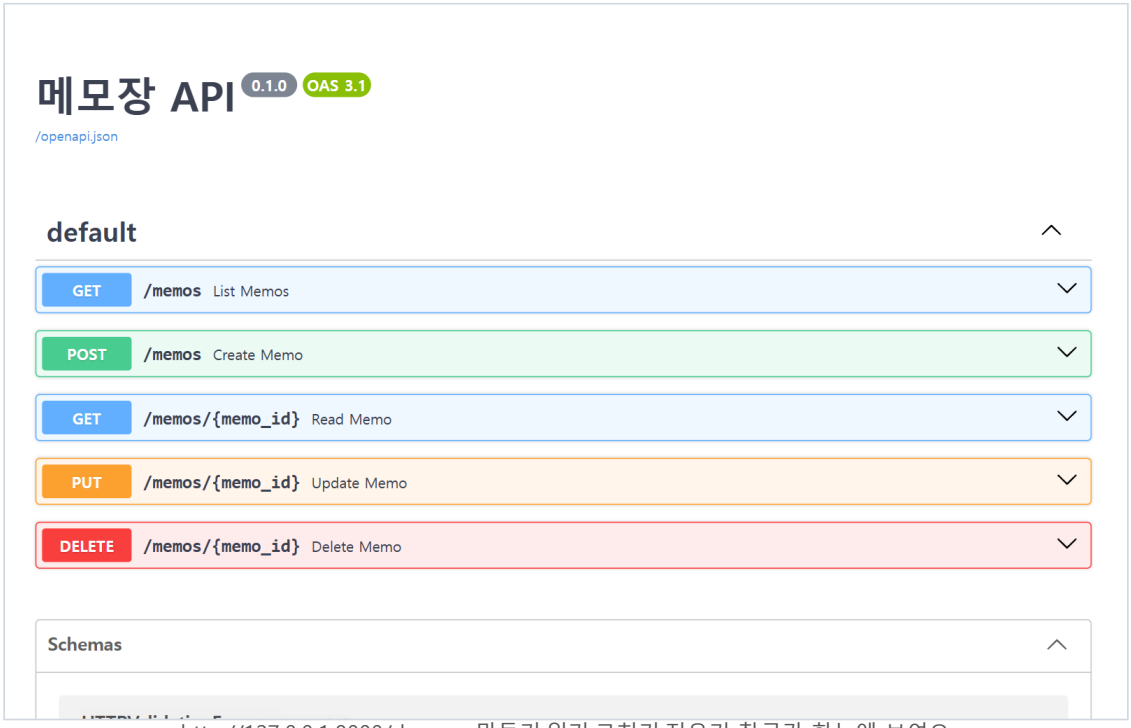
HTTPException 쓰는 법

```
raise HTTPException(status_code=404, detail="9번 메모가 없어요")
```

`raise` 는 "여기서 멈추고 이 오류를 보내요" 라는 뜻이에요. 그 아래 코드는 실행되지 않아요. `detail` 에 적은 글자가 응답에 그대로 나가요.

4. /docs 에서 차례로 시험하기

> **직접 해 보기 | 메모장 API 를 위에서부터 차례로**
>
> - `cd 03_메모장_API` → `uvicorn ex01_memo:app --reload` → `http://127.0.0.1:8000/docs`
> - **POST /memos** 로 메모를 두세 개 만들어요. 응답의 `id` 를 기억해 두세요.
> - **GET /memos** 로 목록을 봐요.
> - **GET /memos/{memo_id}** 에 있는 번호와 없는 번호(예: 9)를 넣어 봐요. 없는 번호는 404 예요.
> - **PUT** 으로 1번 메모를 고치고, **DELETE** 로 2번 메모를 지운 뒤 다시 목록을 봐요.
> - 서버를 껐다 켜면 메모가 모두 사라져요. 딕셔너리는 **서버가 켜져 있는 동안만** 기억하기 때문이에요. (7장에서 파일에 저장해 봐요)

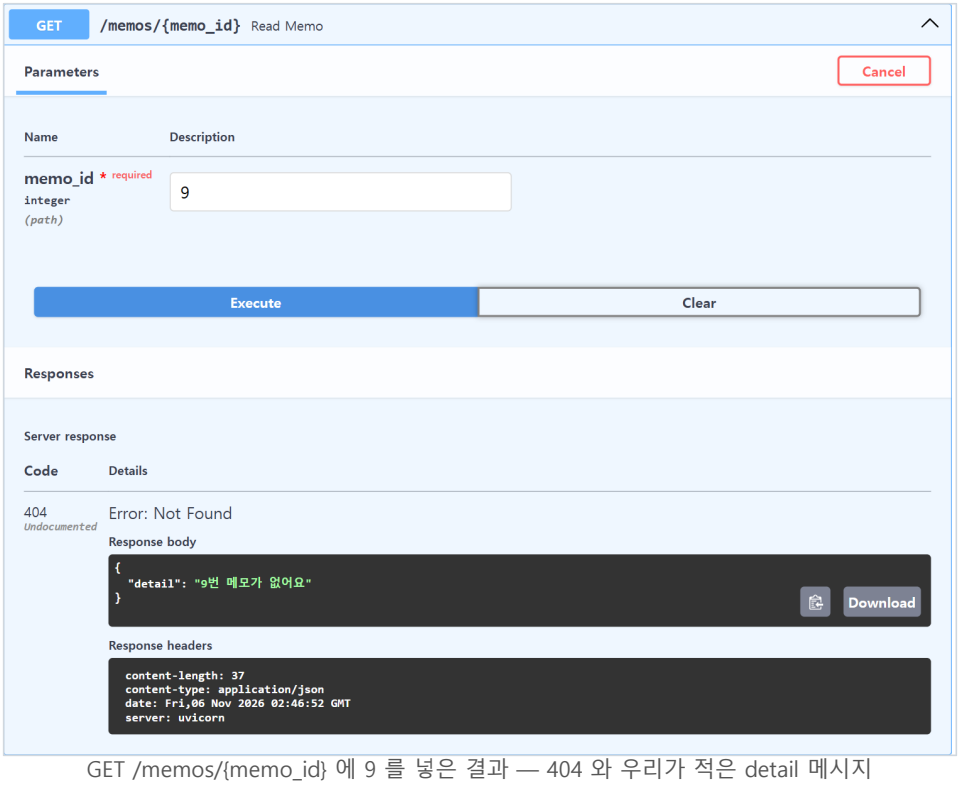


http://127.0.0.1:8000/docs — 만들기·읽기·고치기·지우기 창구가 한눈에 보여요

같은 순서로 요청을 보낸 실제 결과예요.

▶ 메모장 API 에 차례로 보낸 요청과 응답

```
POST /memos  →  201
  {"id": 1, "text": "우유 사기"}
POST /memos  →  201
  {"id": 2, "text": "FastAPI 3장 복습"}
GET /memos  →  200
  [{"id": 1, "text": "우유 사기"}, {"id": 2, "text": "FastAPI 3장 복습"}]
GET /memos/9  →  404
  {"detail": "9번 메모가 없어요"}
PUT /memos/1  →  200
  {"id": 1, "text": "우유 2개 사기"}
DELETE /memos/2  →  200
  {"message": "2번 메모를 지웠어요"}
GET /memos  →  200
  [{"id": 1, "text": "우유 2개 사기"}]
POST /memos  →  422
  {"detail": [{"type": "string_too_short", "loc": ["body", "text"], "msg": "String should have at least 1
character", "input": "", "ctx": {"min_length": 1}}]}
```



GET /memos/{memo_id} 에 9 를 넣은 결과 — 404 와 우리가 적은 detail 메시지

> **알아 두기 | 마지막 요청은 왜 422 일까?**
> `text` 에 빈 글자(`""`)를 보냈어요. `MemoIn` 에 `min_length=1` 규칙을 정해 두어서 2장처럼 Pydantic 이 막았어요.

5. 자주 막히는 곳과 정리

증상	원인	해결
메모를 만들었는데 목록이 비어 있어요	`--reload` 로 코드를 저장하면 서버가 다시 켜지면서 딕셔너리가 비워져요	코드를 다 고친 뒤 메모를 만들기
`UnboundLocalError: next_id`	함수 안에서 `global next_id` 를 빠뜨림	함수 첫 줄에 `global next_id`
405 Method Not Allowed	주소는 맞는데 메서드가 다름 (DELETE 를 GET 으로)	`/docs` 에서 메서드 확인
404 `{"detail":"Not Found"}`	주소 자체가 틀림 (우리가 쓴 detail 이 아님)	주소 철자 확인

배운 것	한 줄 정리
CRUD	만들기 POST · 읽기 GET · 고치기 PUT · 지우기 DELETE
만들기 성공	`status_code=201`
오류 응답	`raise HTTPException(status_code=404, detail="...")`
저장	딕셔너리 — 서버를 끄면 사라져요

연습문제

문제 1. 메모장 API 에 검색 기능을 더하세요. `GET /search?keyword=우유` → `keyword` 가 들어간 메모만 리스트로 돌려주기.

문제 2. 장바구니 API 를 만드세요. `POST /cart {"item": "사과", "count": 3}` 으로 담기(201), `GET /cart` 로 담긴 목록과 전체 개수, `DELETE /cart/{item}` 으로 빼기(없으면 404).

연습문제 정답 및 해설

문제 1 정답

문제1.py

```python
# 문제 1. 메모장 API 에 "글자로 검색" 기능을 더하세요.
#        GET /search?keyword=우유 → keyword 가 들어간 메모만 리스트로 돌려주기
# 실행: 이 폴더에서  uvicorn 문제1:app --reload  →  /docs 에서 메모를 몇 개 만든 뒤 /search 시험
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()


class MemoIn(BaseModel):
    text: str = Field(min_length=1, max_length=100)


memos = {}
next_id = 1


@app.post("/memos", status_code=201)
def create_memo(memo: MemoIn):
    global next_id
    memos[next_id] = memo.text
    result = {"id": next_id, "text": memo.text}
    next_id += 1
    return result


# 새로 만든 검색 기능: keyword 가 들어간 메모만 골라 담기
@app.get("/search")
def search(keyword: str):
    return [{"id": i, "text": t} for i, t in memos.items() if keyword in t]
```

▶ 실행 결과

```
POST /memos  →  201
  {"id": 1, "text": "우유 사기"}
POST /memos  →  201
  {"id": 2, "text": "숙제하기"}
POST /memos  →  201
  {"id": 3, "text": "우유 2개 더"}
GET /search?keyword=우유  →  200
  [{"id": 1, "text": "우유 사기"}, {"id": 3, "text": "우유 2개 더"}]
```

`if keyword in t` 는 "글자 t 안에 keyword 가 들어 있으면" 이라는 파이썬 문법이에요.

문제 2 정답

문제2.py

```python
# 문제 2. 장바구니 API 를 만드세요. (메모장 API 와 같은 모양이에요)
#        POST /cart  {"item": "사과", "count": 3}  → 담기 (201)
#        GET  /cart                                → 담긴 목록과 전체 개수
#        DELETE /cart/{item}                       → 그 상품 빼기 (없으면 404)
# 실행: 이 폴더에서  uvicorn 문제2:app --reload  →  /docs 에서 시험
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI()
```

문제2.py

```
10
11
12 class CartIn(BaseModel):
13     item: str = Field(min_length=1)
14     count: int = Field(default=1, ge=1, le=99)
15
16
17 cart = {}                                    # 상품 이름 → 개수
18
19
20 @app.post("/cart", status_code=201)
21 def add_to_cart(c: CartIn):
22     cart[c.item] = cart.get(c.item, 0) + c.count   # 이미 있으면 개수를 더해요
23     return {"item": c.item, "count": cart[c.item]}
24
25
26 @app.get("/cart")
27 def show_cart():
28     return {"items": cart, "total_count": sum(cart.values())}
29
30
31 @app.delete("/cart/{item}")
32 def remove_from_cart(item: str):
33     if item not in cart:
34         raise HTTPException(status_code=404, detail=f"장바구니에 {item} 이(가) 없어요")
35     del cart[item]
36     return {"message": f"{item} 을(를) 뺐어요"}
```

▶ 실행 결과

```
POST /cart  →  201
  {"item": "사과", "count": 3}
POST /cart  →  201
  {"item": "우유", "count": 1}
POST /cart  →  201
  {"item": "사과", "count": 5}
GET /cart  →  200
  {"items": {"사과": 5, "우유": 1}, "total_count": 6}
DELETE /cart/배  →  404
  {"detail": "장바구니에 배 이(가) 없어요"}
DELETE /cart/우유  →  200
  {"message": "우유 을(를) 뺐어요"}
GET /cart  →  200
  {"items": {"사과": 5}, "total_count": 5}
```

`cart.get(c.item, 0)` 은 "있으면 그 개수, 없으면 0" 이에요. 같은 상품을 또 담으면 개수가 더해져요.

04장

웹 페이지와 API

HTML 화면에서 버튼으로 API 를 부르고, 동기 · 비동기를 체험해요

항목	내용
예제 폴더	`fastapi/초급과정/04_웹페이지와_API`
예제 파일	`ex01_html.py` ~ `ex03_post.py` + `page1~3.html`
예상 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- 서버가 HTML 화면을 돌려주게 할 수 있어요.
- 버튼을 누르면 `fetch` 로 API 를 부르고, 받은 값을 화면에 보여 줄 수 있어요.
- 동기와 비동기의 차이를 직접 보고 설명할 수 있어요.
- 입력 칸의 값을 POST 로 보내고, 오류를 화면에 보여 줄 수 있어요.

도움말 | 이번 장의 구성

- 1. 웹 페이지와 API 는 어떻게 함께 일할까?
- 2. 1단계 · HTML 페이지 돌려주기
- 3. 2단계 · 버튼으로 API 부르기 — fetch 와 동기 · 비동기
- 4. 3단계 · 입력 칸으로 POST 보내기
- 5. 정리
- 연습문제 · 정답 및 해설

1. 웹 페이지와 API 는 어떻게 함께 일할까?

지금까지는 `/docs` 로 API 를 불렀어요. 실제 서비스에서는 **웹 페이지**가 API 를 불러요. 버튼을 누르면 페이지 속 **자바스크립트**가 서버에 요청을 보내고, 받은 JSON 으로 화면을 바꿔요. 이번 장은 이 흐름을 **세 단계**로 천천히 익혀요.

단계	예제	새로 배우는 것
1단계	`ex01_html.py` + `page1.html`	서버가 JSON 대신 **HTML(화면)** 을 돌려주기
2단계	`ex02_fetch.py` + `page2.html`	버튼 → `fetch` 로 **GET 요청**, 그리고 **동기 vs 비동기** 체험
3단계	`ex03_post.py` + `page3.html`	입력 칸 → **POST 요청**, 오류(422)를 화면에 보여 주기

주의 | 서버는 반드시 이 폴더에서 켜요

모든 단계는 `04_웹페이지와_API` 폴더로 `cd` 한 뒤 켜요. 단계를 바꿀 때는 켜 둔 서버를 `Ctrl + C` 로 끄고 다음 파일로 다시 켜요. 다른 폴더에서 켜면 `Could not import module` 오류가 나요.

알아 두기 | HTML 과 자바스크립트를 처음 봐도 괜찮아요

HTML 은 화면의 **뼈대**(제목, 글, 버튼), 자바스크립트는 화면의 **움직임**(버튼을 누르면 할 일)이에요. 이 장에서는 꼭 필요한 몇 가지만 표로 정리해 두었어요. 자세한 문법은 HTML·CSS 과정에서 배워요.

2. 1단계 · HTML 페이지 돌려주기

브라우저는 서버가 보내 준 내용이 **JSON 이면 글자 그대로**, **HTML 이면 화면으로 그려서** 보여 줘요. 서버가 "이건 HTML 이에요" 라고 알려 주는 방법 두 가지를 비교해 봐요.

예제 파일 `fastapi/초급과정/04_웹페이지와_API/ex01_html.py`

ex01_html.py

```python
# 4장 1단계: 서버가 JSON 대신 HTML(웹 페이지)을 돌려주기 (서버로 실행)
# 실행: 이 폴더에서  uvicorn ex01_html:app --reload
# 확인: 브라우저에서 아래 세 주소를 차례로 열어 보고, 보이는 모양이 어떻게 다른지 비교해요
#   http://127.0.0.1:8000/api/hello   → JSON (데이터)
#   http://127.0.0.1:8000/inline      → 코드 안에 쓴 HTML (화면)
#   http://127.0.0.1:8000             → HTML 파일 page1.html (화면)
from pathlib import Path

from fastapi import FastAPI
from fastapi.responses import FileResponse, HTMLResponse

app = FastAPI()
HERE = Path(__file__).parent                  # 이 파일이 있는 폴더 (HTML 파일을 찾을 때 기준)


# ① 지금까지 만든 것과 같은 API: 딕셔너리 → JSON
@app.get("/api/hello")
def hello():
    return {"message": "안녕하세요, 저는 JSON 이에요"}


# ② HTML 을 글자로 직접 돌려주기
# response_class=HTMLResponse: "이 글자는 JSON 이 아니라 HTML 이에요" 라고 브라우저에 알려 줘요
# (이 표시가 없으면 브라우저는 태그를 그림으로 그리지 않고 글자 그대로 보여 줘요)
@app.get("/inline", response_class=HTMLResponse)
def inline_page():
    return "<h1>코드 안에 쓴 HTML</h1><p>태그가 <b>화면</b>으로 바뀌어 보여요.</p>"


# ③ HTML 파일을 통째로 돌려주기 — 화면이 길어지면 이 방법이 편해요
# include_in_schema=False: 화면용 주소라서 /docs API 문서에는 보이지 않게
@app.get("/", include_in_schema=False)
def page():
    return FileResponse(HERE / "page1.html")
```

예제 파일 `fastapi/초급과정/04_웹페이지와_API/page1.html`

page1.html

```html
<!DOCTYPE html>
<!-- 4장 1단계 화면: 자바스크립트 없이 HTML 태그만 쓴 가장 단순한 페이지 -->
<html lang="ko">
<head>
  <!-- head: 화면에 보이지 않는 정보 (글자 인코딩, 탭 제목, 디자인) -->
  <meta charset="UTF-8">
  <title>1단계 - HTML 페이지</title>
  <style>
    /* style: 글자 모양·색·간격을 정하는 CSS (HTML·CSS 과정에서 자세히) */
    body { font-family: "Malgun Gothic", sans-serif; max-width: 560px; margin: 40px auto; padding: 0 16px; }
    .box { background: #eef4fb; border-left: 4px solid #2e75b6; padding: 10px 14px; }
    code { color: #c7254e; }
  </style>
</head>
<body>
```

page1.html

```
  <!-- body: 화면에 보이는 내용 -->
  <h1>안녕하세요, 첫 웹 페이지예요</h1>
  <p>이 화면은 FastAPI 서버가 <b>page1.html</b> 파일을 그대로 보내 준 거예요.</p>

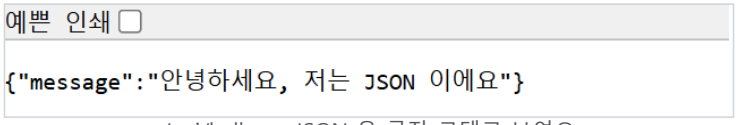
  <h2>주소마다 돌려주는 것이 달라요</h2>
  <ul>
    <!-- a 태그: 누르면 그 주소로 이동하는 링크 -->
    <li><a href="/api/hello">/api/hello</a> — 데이터(JSON)를 돌려줘요</li>
    <li><a href="/inline">/inline</a> — 코드 안에 쓴 HTML 을 돌려줘요</li>
    <li><a href="/docs">/docs</a> — API 문서 (이 페이지 주소 "/" 는 문서에 없어요)</li>
  </ul>

  <div class="box">
    다음 단계에서는 이 페이지에 <b>버튼</b>을 달고, 버튼을 누르면 자바스크립트가
    <code>/api/...</code> 주소로 요청을 보내 받은 데이터를 화면에 보여 줘요.
  </div>
</body>
</html>
```

HTML 태그	뜻
`<head>` / `<body>`	화면에 안 보이는 정보 / 화면에 보이는 내용
`<h1>` `<h2>`	큰 제목, 중간 제목
`<p>`	문단
`<ul>` + `<li>`	점 목록과 그 항목
`<a href="주소">`	누르면 그 주소로 이동하는 링크
`<b>` / `<div>`	굵은 글자 / 영역을 묶는 상자



http://127.0.0.1:8000 — FileResponse 로 돌려준 page1.html



/api/hello — JSON 은 글자 그대로 보여요



/inline — HTML 태그(h1, p, b)가 화면으로 그려져요

3. 2단계 · 버튼으로 API 부르기 — fetch 와 동기 · 비동기

이제 화면에 버튼을 달고, 버튼을 누르면 **자바스크립트**가 서버에 요청을 보내 받은 JSON 을 화면에 써 넣어요.
요청을 보내는 함수 이름이 `fetch`(가져오다) 예요.

예제 파일 `fastapi/초급과정/04_웹페이지와_API/ex02_fetch.py`

ex02_fetch.py

```python
# 4장 2단계: 버튼을 누르면 자바스크립트가 API 를 부르기 + 동기/비동기 체험 (서버로 실행)
# 실행: 이 폴더에서  uvicorn ex02_fetch:app --reload
# 확인: 브라우저에서 http://127.0.0.1:8000 → 버튼들을 눌러 보세요
import time
from datetime import datetime
from pathlib import Path

from fastapi import FastAPI
from fastapi.responses import FileResponse

app = FastAPI()
HERE = Path(__file__).parent


# 바로 대답하는 API: 지금 시각
@app.get("/api/time")
def get_time():
    return {"now": datetime.now().strftime("%H:%M:%S")}


# 이름을 쿼리로 받아 인사하는 API (1장의 쿼리와 같아요)
@app.get("/api/hello")
def hello(name: str = "손님"):
    return {"message": f"{name}님, 반가워요!"}


# 일부러 느리게 대답하는 API: 모델 예측·데이터베이스처럼 "시간이 걸리는 일" 을 흉내 내요
# seconds 쿼리로 기다릴 시간을 정해요 (0 ~ 5초)
@app.get("/api/slow")
def slow(seconds: float = 2.0):
    seconds = max(0.0, min(seconds, 5.0))
    time.sleep(seconds)                          # 서버 쪽에서 기다리는 시간
    return {"message": f"{seconds}초 걸리는 일을 마쳤어요", "finished": datetime.now().strftime("%H:%M:%S")}


@app.get("/", include_in_schema=False)
def page():
    return FileResponse(HERE / "page2.html")
```

예제 파일 `fastapi/초급과정/04_웹페이지와_API/page2.html`

page2.html

```html
<!DOCTYPE html>
<!-- 4장 2단계 화면: 버튼 → fetch 로 API 호출 → 받은 JSON 을 화면에 표시 + 동기/비동기 체험 -->
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>2단계 - fetch 와 비동기</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 600px; margin: 30px auto; padding: 0 16px;
}
    section { background: #f5f7fb; border-radius: 8px; padding: 12px 16px; margin-bottom: 14px; }
    h2 { font-size: 17px; margin: 4px 0 10px; }
    button { padding: 6px 12px; font-size: 14px; margin: 2px 4px 2px 0; }
```

page2.html

```
    input { padding: 6px; font-size: 14px; width: 120px; }
    .result { margin-top: 8px; min-height: 22px; color: #1f4e79; font-weight: bold; }
    #log { background: #1e1e1e; color: #e6e6e6; font-family: Consolas, monospace; font-size: 13px;
           padding: 8px 10px; min-height: 90px; white-space: pre-line; border-radius: 6px; }
    #counter { font-size: 20px; font-weight: bold; color: #c55a11; }
  </style>
</head>
<body>
  <h1>2단계 · 버튼으로 API 부르기</h1>

  <!-- ① 가장 단순한 fetch: 버튼 → GET /api/time → 화면에 표시 -->
  <section>
    <h2>① 지금 시각 가져오기 (GET)</h2>
    <button id="time-btn">시각 가져오기</button>
    <div class="result" id="time-result"></div>
  </section>

  <!-- ② 입력값을 쿼리로 실어 보내기: GET /api/hello?name=... -->
  <section>
    <h2>② 이름 보내기 (쿼리)</h2>
    <input id="name" value="철수">
    <button id="hello-btn">인사 받기</button>
    <div class="result" id="hello-result"></div>
  </section>

  <!-- ③ 동기 vs 비동기: 기다리는 동안 화면이 멈추는지 직접 확인 -->
  <section>
    <h2>③ 동기 vs 비동기 체험</h2>
    <p>
      먼저 <b>[느린 요청]</b> 을 누르고, 응답이 올 때까지 <b>[클릭 +1]</b> 을 계속 눌러 보세요. 숫자가 올라가요(비동기).<br>
      다음에 <b>[화면 멈추기]</b> 를 누르고 [클릭 +1] 을 눌러 보세요. 2초 동안 아무 반응이 없어요(동기).
    </p>
    <button id="slow-btn">느린 요청 (2초, 비동기)</button>
    <button id="block-btn">화면 멈추기 (2초, 동기)</button>
    <button id="count-btn">클릭 +1</button>
    클릭 수: <span id="counter">0</span>
    <div id="log"></div>
  </section>

  <script>
    // ── 도우미: 화면의 로그 창에 한 줄 쓰기 (시각과 함께) ──
    function log(text) {
      const time = new Date().toLocaleTimeString();
      document.getElementById("log").textContent += `[${time}] ${text}\n`;
    }

    // ── ① 버튼 → GET 요청 ──
    // async function: 안에서 await 를 쓸 수 있는 함수
    // await: "응답이 올 때까지 이 함수만 잠깐 기다려요" — 그동안 화면(다른 버튼)은 계속 움직여요
    async function loadTime() {
      const res = await fetch("/api/time");          // 서버에 GET 요청 → 응답(Response) 받기
      const data = await res.json();                 // 응답 본문 JSON → 자바스크립트 객체
      document.getElementById("time-result").textContent = `서버 시각: ${data.now}`;
    }
    // 버튼을 눌렀을 때(onclick) 위 함수를 실행하도록 연결
    document.getElementById("time-btn").onclick = loadTime;

    // ── ② 입력값을 쿼리로 보내기 ──
    document.getElementById("hello-btn").onclick = async () => {
      const name = document.getElementById("name").value;
      // encodeURIComponent: 한글·띄어쓰기를 주소에 안전한 글자로 바꿔요 (0장 2절 URL)
```

page2.html

```
      const res = await fetch(`/api/hello?name=${encodeURIComponent(name)}`);
      const data = await res.json();
      document.getElementById("hello-result").textContent = data.message;
    };

    // ── ③-1 비동기: 느린 요청을 보내 놓고, 기다리는 동안 다른 일을 할 수 있어요 ──
    document.getElementById("slow-btn").onclick = async () => {
      log("1) 느린 요청을 보냈어요 (기다리는 동안 [클릭 +1] 을 눌러 보세요)");
      const res = await fetch("/api/slow?seconds=2");  // 여기서 이 함수만 기다리고, 화면은 계속 움직여요
      const data = await res.json();
      log(`3) 응답 도착: ${data.message}`);
    };

    // ── ③-2 동기: 자바스크립트가 2초 동안 계산만 하며 붙잡고 있으면 화면 전체가 멈춰요 ──
    document.getElementById("block-btn").onclick = () => {
      log("화면을 2초 동안 멈춰요 (이 동안 버튼이 반응하지 않아요)");
      const end = Date.now() + 2000;
      while (Date.now() < end) { }                    // 아무 일도 안 하면서 2초를 꽉 붙잡기
      log("멈춤이 풀렸어요 — 그동안 누른 클릭은 이제야 처리돼요");
    };

    // ── 클릭 수 세기: 화면이 살아 있는지 확인하는 용도 ──
    let count = 0;
    document.getElementById("count-btn").onclick = () => {
      count += 1;
      document.getElementById("counter").textContent = count;
      log(`2) 클릭 +1 → ${count}`);
    };
  </script>
</body>
</html>
```

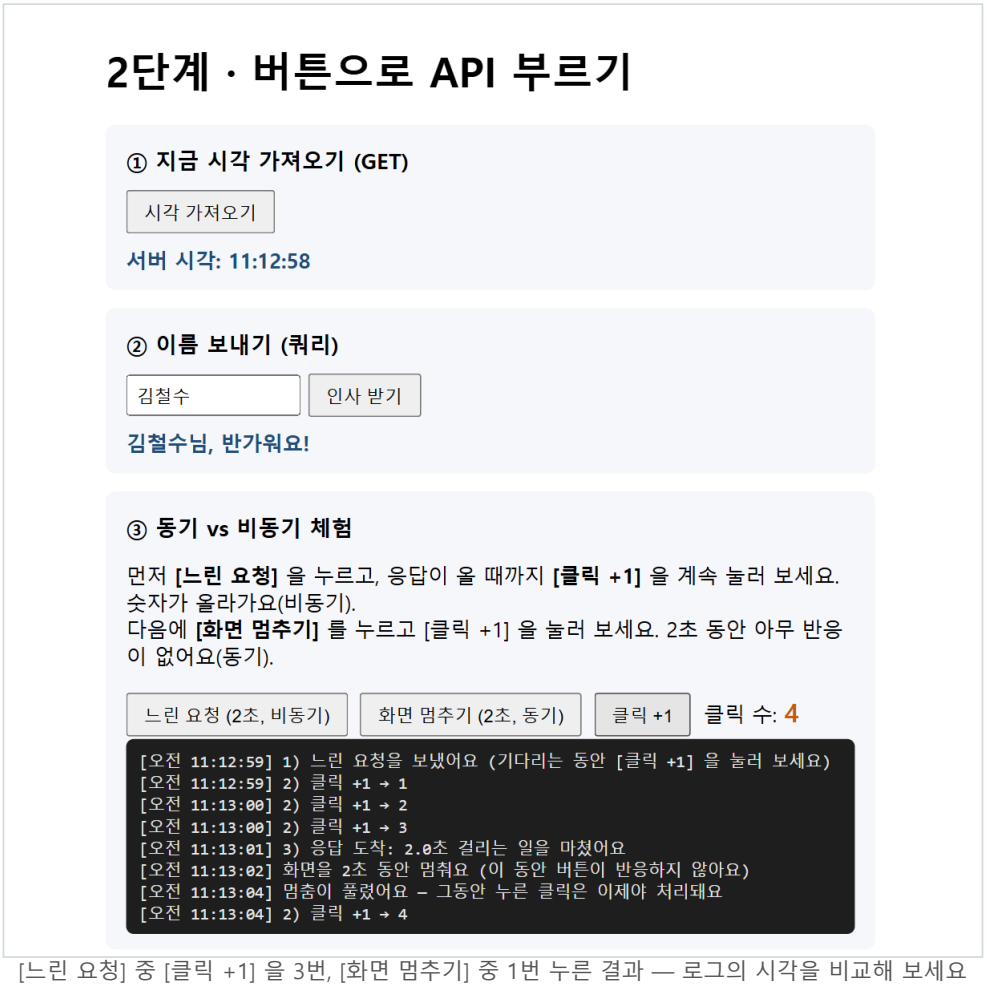
자바스크립트	뜻
`document.getElementById("time-btn")`	화면에서 `id="time-btn"` 인 요소 찾기
`요소.onclick = 함수`	그 요소를 누르면 함수 실행
`요소.textContent = "글자"`	요소 안의 글자 바꾸기
`await fetch("/api/time")`	서버에 GET 요청을 보내고 응답이 올 때까지 기다리기
`await res.json()`	응답 JSON 을 자바스크립트 값으로 바꾸기
`${data.now}`	백틱(키보드 1 왼쪽 기호)으로 감싼 글자 안에 값 넣기 — 파이썬의 f-문자열과 같아요
`async () => { ... }`	안에서 `await` 를 쓸 수 있는 함수

비유로 이해하기 | 동기와 비동기 — 카페 주문으로 이해하기

동기는 주문한 커피가 나올 때까지 계산대 앞에 서서 아무것도 못 하는 것이고, **비동기**는 진동벨을 받아 자리에 앉아 다른 일을 하다가 벨이 울리면 커피를 받는 거예요. `await fetch(...)` 는 진동벨이에요. 응답을 기다리는 동안에도 화면의 다른 버튼은 계속 움직여요.

직접 해 보기 | ③ 동기 vs 비동기 직접 체험하기

- `uvicorn ex02_fetch:app --reload` 로 켜고 `http://127.0.0.1:8000` 에 접속해요.
- **[느린 요청]** 을 누르고 2초가 지나기 전에 **[클릭 +1]** 을 여러 번 눌러요. 숫자가 바로바로 올라가요. 응답을 기다리는 **동안에도** 다른 일을 한 거예요(비동기).
- 이번에는 **[화면 멈추기]** 를 누르고 바로 [클릭 +1] 을 눌러요. 2초 동안 숫자가 꿈쩍도 안 하다가 멈춤이 풀린 뒤에야 올라가요. 화면 전체가 멈춘 거예요(동기).



[느린 요청] 중 [클릭 +1] 을 3번, [화면 멈추기] 중 1번 누른 결과 — 로그의 시각을 비교해 보세요

알아 두기 | 로그 읽기 — 왜 이 순서일까?

느린 요청을 보낸 시각(59초)과 응답이 도착한 시각(01초) **사이에** 클릭 1 · 2 · 3 이 끼어 있어요. `await` 가 기다리는 동안 화면을 놓아 주었기 때문이에요. [화면 멈추기] 는 화면을 놓아 주지 않아서, 그동안 누른 클릭이 멈춤이 풀린 04초에야 처리됐어요. 웹 페이지가 느린 서버를 기다릴 때 화면이 멈추지 않는 건 이 비동기 덕분이에요.

4. 3단계 · 입력 칸으로 POST 보내기

이번에는 입력 칸에 적은 값을 **POST 요청의 본문(JSON)** 으로 보내요. 2장의 `/docs` Try it out 에서 하던 일을 웹 페이지가 대신 하는 거예요.

예제 파일 fastapi/초급과정/04_웹페이지와_API/ex03_post.py

ex03_post.py

```python
# 4장 3단계: 입력 칸에 적은 값을 POST 로 보내고, 결과·오류를 화면에 보여 주기 (서버로 실행)
# 실행: 이 폴더에서  uvicorn ex03_post:app --reload
# 확인: 브라우저에서 http://127.0.0.1:8000 → 이름과 점수를 넣고 [학점 계산] (점수 150 도 넣어 보세요)
from pathlib import Path

from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

app = FastAPI()
HERE = Path(__file__).parent


# 2장에서 배운 Pydantic 모델: 화면에서 보낸 JSON 의 모양과 규칙
class ScoreIn(BaseModel):
    name: str = Field(min_length=1, max_length=20)    # 이름: 1 ~ 20글자
    score: int = Field(ge=0, le=100)                  # 점수: 0 ~ 100


# POST /api/grade: 본문 JSON {"name": ..., "score": ...} → 학점 계산
@app.post("/api/grade")
def grade(data: ScoreIn):
    if data.score >= 90:
        letter = "A"
    elif data.score >= 80:
        letter = "B"
    elif data.score >= 70:
        letter = "C"
    else:
        letter = "F"
    return {"name": data.name, "score": data.score, "grade": letter}


@app.get("/", include_in_schema=False)
def page():
    return FileResponse(HERE / "page3.html")
```

예제 파일 fastapi/초급과정/04_웹페이지와_API/page3.html

page3.html

```html
<!DOCTYPE html>
<!-- 4장 3단계 화면: 입력 칸(form) → POST + JSON 본문 → 성공이면 결과, 422 면 오류 이유 표시 -->
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>3단계 - POST 보내기</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 520px; margin: 40px auto; padding: 0 16px;
}
    form { display: flex; gap: 8px; margin-bottom: 14px; }
    input { padding: 8px; font-size: 15px; }
    input[type=number] { width: 90px; }
    button { padding: 8px 14px; font-size: 15px; }
    #result { padding: 12px; border-radius: 6px; min-height: 22px; }
    .ok { background: #e2f0d9; color: #385723; }
    .error { background: #fce4d6; color: #843c0c; }
    pre { background: #1e1e1e; color: #e6e6e6; padding: 10px; border-radius: 6px; font-size: 13px; }
  </style>
</head>
<body>
```

page3.html

```
  <h1>3단계 · 학점 계산기 (POST)</h1>

  <!-- form: 입력 칸들을 묶는 태그. [학점 계산] 버튼(type=submit)을 누르면 submit 이벤트가 일어나요 -->
  <form id="grade-form">
    <input id="name" placeholder="이름" value="영희">
    <input id="score" type="number" placeholder="점수" value="87">
    <button type="submit">학점 계산</button>
  </form>

  <div id="result"></div>
  <p>서버에 실제로 보낸 JSON 본문:</p>
  <pre id="sent">(아직 보내지 않았어요)</pre>

  <script>
    document.getElementById("grade-form").onsubmit = async (event) => {
      // form 은 원래 제출하면 페이지를 새로 불러와요 → 그 기본 동작을 막고 fetch 로 직접 보내요
      event.preventDefault();

      // ① 보낼 데이터 만들기: 입력 칸의 값 → 자바스크립트 객체
      const body = {
        name: document.getElementById("name").value,
        score: Number(document.getElementById("score").value),   // 입력 칸 값은 글자라서 숫자로 바꿔요
      };
      // JSON.stringify: 객체 → JSON 글자 (파이썬의 json.dumps 와 같아요)
      document.getElementById("sent").textContent = JSON.stringify(body, null, 2);

      // ② POST 요청: method, headers(본문이 JSON 이라고 알리기), body(JSON 글자)
      const res = await fetch("/api/grade", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(body),
      });
      const data = await res.json();

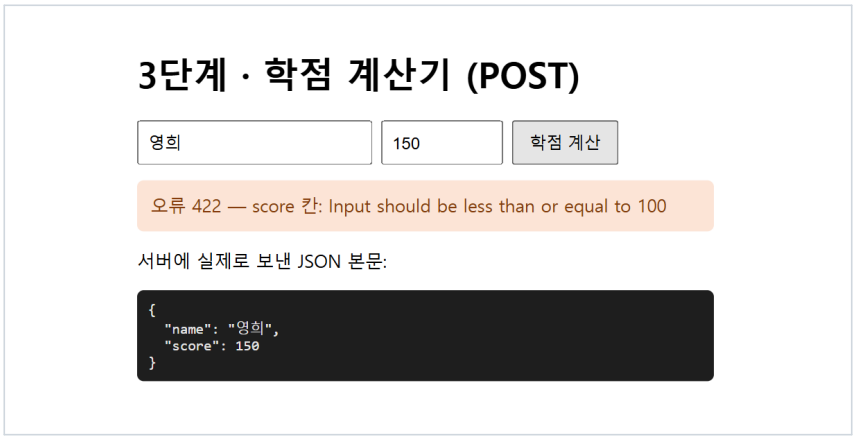
      // ③ 상태 코드에 따라 다르게 보여 주기 — res.ok 는 200번대 성공이면 true
      const box = document.getElementById("result");
      if (res.ok) {
        box.className = "ok";
        box.textContent = `${data.name}님의 점수 ${data.score}점 → 학점 ${data.grade}`;
      } else {
        // 422: Pydantic 규칙에 맞지 않음 → detail[0] 에 어느 칸(loc)이 왜(msg) 틀렸는지 들어 있어요
        const err = data.detail[0];
        box.className = "error";
        box.textContent = `오류 ${res.status} — ${err.loc.at(-1)} 칸: ${err.msg}`;
      }
    };
  </script>
</body>
</html>
```

코드	뜻
`event.preventDefault()`	form 이 페이지를 새로 불러오는 기본 동작을 막기 (그래야 fetch 로 보낼 수 있어요)
`Number(...)`	입력 칸의 값은 항상 글자 → 숫자로 바꾸기
`JSON.stringify(body)`	자바스크립트 값 → JSON 글자
`method: "POST"`	요청 방식 (안 쓰면 GET)
`headers: {"Content-Type": "application/json"}`	"본문이 JSON 이에요" 라고 서버에 알리기

코드	뜻
`res.ok` / `res.status`	성공(200번대)이면 `true` / 상태 코드 숫자



87점 → 학점 B. 아래에 실제로 보낸 JSON 본문이 보여요



점수 150 → 서버가 422 로 막고, 화면에 이유를 보여 줘요

> **도움말 | 세 가지 POST 를 비교해 보세요**
> **2장** `/docs` 에서 JSON 을 손으로 써서 보냈고, **4장** 웹 페이지가 입력 칸 값으로 JSON 을 만들어 보냈어요. 서버 코드(`@app.post` + `BaseModel`)는 똑같아요. **서버는 누가 보냈는지 상관없이 같은 방법으로 받아요.**

5. 정리

배운 것	한 줄 정리
HTML 돌려주기	`return FileResponse("파일.html")` 또는 `HTMLResponse`
fetch GET	`const res = await fetch("/주소"); const data = await res.json();`
fetch POST	`fetch("/주소", {method: "POST", headers: {...}, body: JSON.stringify(값)})`
비동기	`await` 로 기다리는 동안에도 화면은 계속 움직여요
오류 표시	`res.ok` 가 아니면 `data.detail[0].msg` 를 화면에 보여 주기

연습문제

문제 1. 2단계 페이지(`page2.html`)의 [시각 가져오기] 버튼 아래에, 누를 때마다 **몇 번째로 가져왔는지**("3번째 확인") 함께 보여 주도록 고쳐 보세요. (힌트: 클릭 수 세기 코드를 참고)

문제 2. 3단계 페이지에서 점수 칸을 비우고 [학점 계산] 을 누르면 어떤 일이 일어나는지 확인하고, 그 이유를 설명해 보세요.

연습문제 정답 및 해설

문제 1 정답

`loadTime` 함수 밖에 `let timeCount = 0;` 을 만들고, 함수 안에서 하나씩 늘려 글자에 붙여요.

page2.html 의 loadTime 부분을 이렇게 바꿔요

```
let timeCount = 0;                                    // 몇 번 가져왔는지 세기
async function loadTime() {
  const res = await fetch("/api/time");
  const data = await res.json();
  timeCount += 1;
  document.getElementById("time-result").textContent = `서버 시각: ${data.now} (${timeCount}번째 확인)`;
}
```

문제 2 정답

점수 칸이 비면 `Number("")` 가 `0` 이 되어 **0점 → 학점 F** 가 나와요. 입력 칸이 빈 글자일 때 `Number` 가 0 을 돌려주기 때문이에요. 빈 칸을 막고 싶다면 `<input id="score" type="number" required>` 처럼 `required` 를 붙이거나, 보내기 전에 `if (document.getElementById("score").value === "") { ... }` 로 확인해요.

05장

메모장 웹 앱

3장의 API 와 4장의 화면을 합쳐, 진짜로 쓸 수 있는 메모장을 완성해요

항목	내용
예제 폴더	`fastapi/초급과정/05_메모장_웹앱`
예제 파일	`memo_app.py` + `memo_page.html`
예상 시간	3시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- 서버(API)와 화면(HTML)을 합쳐 웹 서비스를 만들 수 있어요.
- 받은 목록으로 화면을 다시 그릴 수 있어요.
- 화면의 버튼이 어떤 HTTP 요청을 보내는지 설명할 수 있어요.

도움말 | 이번 장의 구성

- 1. 지금까지 만든 것 합치기
- 2. 서버 — memo_app.py
- 3. 화면 — memo_page.html
- 4. 써 보기
- 5. 정리
- 연습문제 · 정답 및 해설

1. 지금까지 만든 것 합치기

3장에서 **메모장 API**(서버 쪽)를, 4장에서 **API 를 부르는 웹 페이지**(화면 쪽)를 만들었어요. 이 둘을 합치면 진짜로 쓸 수 있는 **메모장 웹 서비스**가 돼요. 새로 배우는 문법은 거의 없어요. **이미 배운 조각을 맞추는 장**이에요.

조각	처음 배운 곳	이 장에서
메모장 API (POST · GET · PUT · DELETE)	3장	`memo_app.py` 아래쪽에 그대로
HTML 돌려주기 (`FileResponse`)	4장 1단계	주소 `/` 에서 `memo_page.html`
버튼 → `fetch` → 화면 바꾸기	4장 2단계	목록 불러오기, 지우기
입력 칸 → POST	4장 3단계	메모 저장
목록으로 화면 그리기	**새로 배움**	`for` 로 메모마다 한 줄씩

2. 서버 — memo_app.py

예제 파일 `fastapi/초급과정/05_메모장_웹앱/memo_app.py`

memo_app.py

```
# 5장 예제: 메모장 웹 앱 — 3장의 메모장 API + 4장의 웹 페이지 = 완성된 웹 서비스
# 실행: 이 폴더에서  uvicorn memo_app:app --reload
# 확인: 브라우저에서 http://127.0.0.1:8000 → 메모를 쓰고, 고치고, 지워 보세요
from pathlib import Path

```

memo_app.py

```python
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

app = FastAPI(title="메모장 웹 앱")
HERE = Path(__file__).parent


# 화면: 주소 "/" 로 들어오면 memo_page.html 을 돌려줘요 (4장 1단계)
@app.get("/", include_in_schema=False)
def page():
    return FileResponse(HERE / "memo_page.html")


# ── 아래는 3장 메모장 API 와 똑같아요 ──
# 메모를 만들거나 고칠 때 받는 모양
class MemoIn(BaseModel):
    text: str = Field(min_length=1, max_length=100)


# 메모를 저장할 곳: 번호 → 메모 글자 (서버를 끄면 사라져요)
memos = {}
next_id = 1                                  # 다음에 만들 메모 번호


# C: 메모 만들기 — status_code=201 은 "새로 만들었어요" 라는 성공 코드
@app.post("/memos", status_code=201)
def create_memo(memo: MemoIn):
    global next_id                           # 함수 밖의 next_id 를 바꾸겠다는 표시
    memos[next_id] = memo.text
    result = {"id": next_id, "text": memo.text}
    next_id += 1
    return result


# R: 메모 전체 보기
@app.get("/memos")
def list_memos():
    return [{"id": i, "text": t} for i, t in memos.items()]


# R: 메모 하나 보기 — 없는 번호면 404 오류
@app.get("/memos/{memo_id}")
def read_memo(memo_id: int):
    if memo_id not in memos:
        # raise HTTPException: "여기서 멈추고 이 오류를 응답으로 보내요"
        raise HTTPException(status_code=404, detail=f"{memo_id}번 메모가 없어요")
    return {"id": memo_id, "text": memos[memo_id]}


# U: 메모 고치기
@app.put("/memos/{memo_id}")
def update_memo(memo_id: int, memo: MemoIn):
    if memo_id not in memos:
        raise HTTPException(status_code=404, detail=f"{memo_id}번 메모가 없어요")
    memos[memo_id] = memo.text
    return {"id": memo_id, "text": memo.text}


# D: 메모 지우기
@app.delete("/memos/{memo_id}")
def delete_memo(memo_id: int):
```

memo_app.py

```
    if memo_id not in memos:
        raise HTTPException(status_code=404, detail=f"{memo_id}번 메모가 없어요")
    del memos[memo_id]
    return {"message": f"{memo_id}번 메모를 지웠어요"}
```

위쪽의 `page()` 함수 하나만 새로 생겼고, 아래쪽은 3장의 메모장 API 를 그대로 옮겨 왔어요.

3. 화면 — memo_page.html

예제 파일 `fastapi/초급과정/05_메모장_웹앱/memo_page.html`

memo_page.html

```
<!DOCTYPE html>
<!-- 5장 메모장 화면: 4장에서 배운 fetch 로 메모장 API(GET · POST · PUT · DELETE)를 불러요 -->
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>나의 메모장</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 520px; margin: 40px auto; padding: 0 16px; background: #fffaf0; }
    form { display: flex; gap: 8px; margin-bottom: 8px; }
    input { flex: 1; padding: 8px; font-size: 15px; }
    button { padding: 6px 12px; font-size: 14px; }
    ul { list-style: none; padding: 0; }
    li { display: flex; align-items: center; gap: 6px; background: white; padding: 10px; margin-bottom: 6px;
         border-radius: 6px; box-shadow: 0 1px 3px rgba(0,0,0,0.1); }
    li span { flex: 1; }
    #msg { color: #c00000; min-height: 20px; }
  </style>
</head>
<body>
  <h1>나의 메모장</h1>

  <form id="memo-form">
    <input id="memo-text" placeholder="메모를 입력하세요">
    <button type="submit">저장</button>
  </form>
  <p id="msg"></p>
  <p>메모 <b id="count">0</b>개</p>
  <ul id="list"></ul>

  <script>
    // ① 목록 불러와서 화면에 그리기 — GET /memos
    async function loadMemos() {
      const res = await fetch("/memos");
      const memos = await res.json();                 // [{id: 1, text: "..."}, ...]
      document.getElementById("count").textContent = memos.length;
      const list = document.getElementById("list");
      list.innerHTML = "";                            // 비우고 처음부터 다시 그리기
      for (const memo of memos) {                     // 메모 하나마다 li 한 줄 만들기
        const li = document.createElement("li");
        li.innerHTML = `<span></span><button>고치기</button><button>지우기</button>`;
        li.querySelector("span").textContent = memo.text;
        const [editBtn, deleteBtn] = li.querySelectorAll("button");
        editBtn.onclick = () => editMemo(memo);
        deleteBtn.onclick = () => deleteMemo(memo.id);
        list.appendChild(li);
      }
```

memo_page.html

```
    }

    // ② 메모 저장 — POST /memos (4장 3단계와 같은 방법)
    document.getElementById("memo-form").onsubmit = async (event) => {
      event.preventDefault();
      const input = document.getElementById("memo-text");
      const res = await fetch("/memos", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text: input.value }),
      });
      if (res.ok) {
        input.value = "";
        document.getElementById("msg").textContent = "";
        loadMemos();
      } else {
        document.getElementById("msg").textContent = "메모를 1 ~ 100글자로 써 주세요";   // 422
      }
    };

    // ③ 메모 고치기 — PUT /memos/번호
    async function editMemo(memo) {
      const newText = prompt("메모를 고쳐 쓰세요", memo.text);   // 작은 입력 창 띄우기
      if (!newText) return;                                      // 취소하면 아무것도 안 해요
      await fetch("/memos/" + memo.id, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text: newText }),
      });
      loadMemos();
    }

    // ④ 메모 지우기 — DELETE /memos/번호
    async function deleteMemo(id) {
      await fetch("/memos/" + id, { method: "DELETE" });
      loadMemos();
    }

    loadMemos();                                          // 페이지가 열리면 목록부터
  </script>
</body>
</html>
```

화면에서 하는 일	자바스크립트	보내는 요청	CRUD
페이지를 열 때 · 바뀔 때마다	`loadMemos()`	`GET /memos`	읽기
[저장] 버튼	`fetch("/memos", {method: "POST", ...})`	`POST /memos`	만들기
[고치기] 버튼	`prompt(...)` 로 새 글 받기 → `fetch("/memos/" + id, {method: "PUT", ...})`	`PUT /memos/1`	고치기
[지우기] 버튼	`fetch("/memos/" + id, {method: "DELETE"})`	`DELETE /memos/2`	지우기

비유로 이해하기 | 목록 그리기 = "비우고, 처음부터 다시 그리기"

`loadMemos()` 는 서버에서 메모 목록을 받아 와서 ① 화면의 목록을 **모두 지우고**(`innerHTML = ""`) ② 메모 하나마다 **한 줄씩 새로 만들어 붙여요**(`for` + `createElement` + `appendChild`). 메모를 저장하거나 지운 뒤에는 이 함수를 다시 불러서 화면을 최신으로 바꿔요. 화이트보드를 지우고 새로 쓰는 것과 같아요.

처음 보는 코드	뜻
`document.createElement("li")`	새 목록 줄(`<li>`) 하나 만들기
`li.innerHTML = ...`	그 줄 안에 들어갈 HTML 넣기
`li.querySelector("span")`	그 줄 안에서 `<span>` 찾기
`list.appendChild(li)`	만든 줄을 목록 맨 아래에 붙이기
`prompt("안내", "기본값")`	작은 입력 창 띄우기. 취소하면 `null`
`memos.length`	목록의 개수 (파이썬의 `len(memos)`)

4. 써 보기

직접 해 보기 | 나의 메모장 써 보기

- `cd 05_메모장_웹앱` → `uvicorn memo_app:app --reload` → `http://127.0.0.1:8000`
- 메모를 세 개 써 보세요. 위의 "메모 3개" 숫자도 같이 바뀌어요.
- [고치기] 를 누르면 작은 창이 떠요. 내용을 바꾸고 확인을 눌러요.
- [지우기] 로 하나를 지워요.
- 빈 칸으로 [저장] 을 누르면 안내 문구가 나와요. 서버(Pydantic)가 422 로 막았기 때문이에요.
- 다른 탭에서 `http://127.0.0.1:8000/memos` 를 열어 보세요. 화면의 메모가 JSON 그대로 보여요. **화면은 이 JSON 을 예쁘게 그려 준 것뿐이에요.**



메모 3개를 쓰고, 첫 메모를 [고치기] 로 바꾼 뒤, 빈 칸으로 [저장] 을 눌러 본 화면

알아 두기 | 서버를 껐다 켜면 메모가 사라져요

메모가 딕셔너리(메모리)에만 있기 때문이에요. 7장 미니 프로젝트에서는 **파일(JSON)에 저장해서** 서버를 꺼도 남게 만들어요.

5. 정리

배운 것	한 줄 정리
웹 서비스 =	서버(API) + 화면(HTML + 자바스크립트)
화면 → 서버	버튼 · 입력 → `fetch` 로 요청
서버 → 화면	받은 JSON 으로 화면 바꾸기 (목록은 비우고 다시 그리기)
확인	화면이 이상하면 `/docs` 나 `/memos` 로 API 부터 확인해 보기

연습문제

문제 1. 메모 옆에 글자 수를 `(12자)` 처럼 표시하세요. (힌트: `memo.text.length`)

문제 2. [모두 지우기] 버튼을 만들고, 누르면 메모가 전부 지워지게 하세요. 서버에 `DELETE /memos` 를 하나 더 만들어야 해요.

연습문제 정답 및 해설

정답 폴더의 `정답_app.py` 와 `정답_page.html` 에 두 문제를 모두 반영했어요. 정답 폴더에서 `uvicorn 정답_app:app --reload` 로 켜 보세요.

문제 1 정답

화면 파일에서 메모 글자를 넣는 한 줄만 바꾸면 돼요. 서버는 고칠 필요가 없어요.

정답_page.html — loadMemos 안

```
li.querySelector("span").textContent = `${memo.text} (${memo.text.length}자)`;
```

문제 2 정답

서버에 "번호 없이 `/memos` 로 오는 DELETE" 창구를 하나 더 만들고, 화면에 버튼과 `fetch` 를 더해요.

정답_app.py — 맨 아래에 추가

```
@app.delete("/memos")
def delete_all():
    count = len(memos)
    memos.clear()                              # 딕셔너리 비우기
    return {"message": f"메모 {count}개를 모두 지웠어요"}
```

정답_page.html — 버튼과 자바스크립트 추가

```
<p>메모 <b id="count">0</b>개 <button id="clear-btn">모두 지우기</button></p>

document.getElementById("clear-btn").onclick = async () => {
  if (!confirm("메모를 모두 지울까요?")) return;     // 실수로 누르지 않게 한 번 더 묻기
  await fetch("/memos", { method: "DELETE" });
  loadMemos();
};
```

▶ 서버 쪽 확인

```
POST /memos  →  201
  {"id": 1, "text": "우유 사기"}
POST /memos  →  201
  {"id": 2, "text": "숙제하기"}
DELETE /memos  →  200
  {"message": "메모 2개를 모두 지웠어요"}
GET /memos  →  200
  []
```

06장

AI 맛보기

학습해 둔 AI 모델을 함수 하나로 불러, 예측 웹 서비스를 만들어요

항목	내용
예제 폴더	`fastapi/초급과정/06_AI_맛보기`
예제 파일	`ai_model.py`, `ex01_try_model.py`, `ex02_predict_app.py` + `predict_page.html`
예상 시간	2시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- AI 모델을 "숫자를 넣으면 답이 나오는 함수" 로 쓸 수 있어요.
- 2장과 같은 방법으로 예측 API 를 만들 수 있어요.
- 4장과 같은 방법으로 예측 화면을 만들 수 있어요.

도움말 | 이번 장의 구성

- 1. AI 모델 = 숫자를 넣으면 답이 나오는 함수
- 2. AI 함수 써 보기 — 서버 없이
- 3. 예측 API 와 예측 화면 만들기
- 4. 정리
- 연습문제 · 정답 및 해설

1. AI 모델 = 숫자를 넣으면 답이 나오는 함수

"AI 모델" 이라고 하면 어렵게 들리지만, 웹 서비스를 만드는 입장에서는 이렇게만 생각하면 돼요.

비유로 이해하기 | AI 모델은 "잘 맞히는 함수"

`predict(공부시간, 수면시간, 휴대폰시간)` 처럼 **숫자를 넣으면 답(합격 확률)이 나오는 함수**예요. 다만 이 함수의 계산 방법은 사람이 정한 게 아니라, 많은 학생의 데이터를 보고 **컴퓨터가 스스로 찾아냈어요**(학습). 우리는 학습이 끝난 함수를 **불러다 쓰기만** 해요.

이 장의 목표는 이 함수를 **2장의 커피 주문 API 처럼** 웹 서비스로 만드는 거예요. AI 를 몰라도 지금까지 배운 것만으로 충분해요.

지금까지	이번 장
커피 주문: 메뉴 · 잔 수 → **가격 계산** → JSON	합격 예측: 공부 · 수면 · 휴대폰 시간 → **AI 함수** → JSON
`PRICE[o.menu] * o.count`	`predict(s.study_h, s.sleep_h, s.phone_h)`

2. AI 함수 써 보기 — 서버 없이

예제 파일 `fastapi/초급과정/06_AI_맛보기/ai_model.py`

ai_model.py

```python
# AI 모델을 "함수 하나" 로 쓰게 해 주는 파일 — 안쪽은 몰라도 괜찮아요!
#
# 사용법:  from ai_model import predict
#          확률 = predict(공부시간, 수면시간, 휴대폰시간)     → 0 ~ 1 사이의 합격 확률
#
# 이 모델은 학생 400명의 생활 시간과 합격 여부를 보고 미리 "학습" 해 둔 거예요.
# 학습 결과(숫자 몇 개)가 model/pass_model.json 에 저장되어 있고, 이 파일은 그 숫자로 계산만 해요.
import json
import math
from pathlib import Path

# 파일을 처음 불러올 때 딱 한 번 학습 결과를 읽어 둬요
_m = json.loads((Path(__file__).parent / "model" / "pass_model.json").read_text(encoding="utf-8"))


def predict(study_h: float, sleep_h: float, phone_h: float) -> float:
    """하루 공부 · 수면 · 휴대폰 시간을 넣으면 합격 확률(0 ~ 1)을 돌려줘요."""
    values = [study_h, sleep_h, phone_h]
    # 학습 때와 같은 기준으로 값을 맞추고(평균을 빼고 나누기), 가중치를 곱해 더해요
    score = _m["bias"]
    for v, mean, std, w in zip(values, _m["mean"], _m["std"], _m["weights"]):
        score += (v - mean) / std * w
    # 점수를 0 ~ 1 사이의 확률로 바꾸기 (시그모이드 함수)
    return 1 / (1 + math.exp(-score))
```

`ai_model.py` 는 **열어 보기만** 하세요. 학습해 둔 숫자(`model/pass_model.json`)를 읽어서 계산하는 파일이에요. 우리가 기억할 것은 맨 위 주석의 사용법 한 줄뿐이에요: `from ai_model import predict`.

예제 파일 `fastapi/초급과정/06_AI_맛보기/ex01_try_model.py`

ex01_try_model.py

```python
# 6장 예제 1: AI 모델을 함수처럼 써 보기 — 서버 없이 파이썬으로 먼저 (python 으로 실행)
# 모델 = "숫자를 넣으면 답(확률)이 나오는 함수" 라고 생각하면 돼요
from ai_model import predict

# 함수 한 번 부르기: 공부 4시간, 수면 7시간, 휴대폰 2시간
p = predict(4, 7, 2)
print(f"합격 확률: {p * 100:.1f}%")

# 공부 시간만 바꿔 가며 불러 보기 (수면 7시간, 휴대폰 2시간은 같게)
print("\n공부 시간이 늘면 합격 확률은?")
for study in [0, 1, 2, 3, 4, 5]:
    p = predict(study, 7, 2)
    label = "합격" if p >= 0.5 else "불합격"          # 50% 이상이면 합격으로 판단
    print(f"  공부 {study}시간 → {p * 100:5.1f}%  {label}")

# 휴대폰 시간이 늘면?
print("\n휴대폰을 오래 쓰면? (공부 2시간, 수면 7시간)")
for phone in [0, 2, 4, 6]:
    print(f"  휴대폰 {phone}시간 → {predict(2, 7, phone) * 100:5.1f}%")
```

▶ 터미널: python ex01_try_model.py

```
합격 확률: 99.3%

공부 시간이 늘면 합격 확률은?
  공부 0시간 →   1.2%  불합격
  공부 1시간 →  11.0%  불합격
  공부 2시간 →  56.6%  합격
  공부 3시간 →  93.2%  합격
```

▶ 터미널: python ex01_try_model.py

```
  공부 4시간 →  99.3%  합격
  공부 5시간 →  99.9%  합격

휴대폰을 오래 쓰면? (공부 2시간, 수면 7시간)
  휴대폰 0시간 →  82.3%
  휴대폰 2시간 →  56.6%
  휴대폰 4시간 →  26.7%
  휴대폰 6시간 →   9.3%
```

> **도움말 | 결과를 보고 이야기해 보세요**
> 공부 시간이 늘수록, 휴대폰 시간이 줄수록 합격 확률이 올라가요. 모델이 데이터에서 이런 **경향**을 배웠다는 뜻이에요. 하지만 연습용으로 만든 데이터라서 실제 시험 결과와는 상관없어요!

3. 예측 API 와 예측 화면 만들기

이제 2장(POST + `BaseModel` + `Field`)과 4장(화면에서 POST 보내기)을 그대로 써서 예측 서비스를 만들어요.

예제 파일 `fastapi/초급과정/06_AI_맛보기/ex02_predict_app.py`

ex02_predict_app.py

```python
# 6장 예제 2: AI 예측 웹 서비스 — 예측 API + 예측 화면
# 실행: 이 폴더에서  uvicorn ex02_predict_app:app --reload
# 확인: 브라우저에서 http://127.0.0.1:8000 (예측 화면) 또는 /docs
from pathlib import Path

from fastapi import FastAPI
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

from ai_model import predict                  # 예제 1에서 써 본 그 함수

app = FastAPI(title="합격 예측 서비스")
HERE = Path(__file__).parent


# 받을 값의 모양과 범위 (2장) — 하루는 24시간이니 그 안의 값만 받아요
class Student(BaseModel):
    study_h: float = Field(ge=0, le=16)       # 공부 시간
    sleep_h: float = Field(ge=0, le=16)       # 수면 시간
    phone_h: float = Field(ge=0, le=24)       # 휴대폰 시간


# 예측 API: 받은 값을 AI 함수에 넣고 결과를 JSON 으로 (2장의 커피 주문과 같은 모양!)
@app.post("/predict")
def predict_api(s: Student):
    p = predict(s.study_h, s.sleep_h, s.phone_h)
    return {"probability": round(p, 3), "label": "합격" if p >= 0.5 else "불합격"}


# 예측 화면 (4장 3단계와 같은 방법)
@app.get("/", include_in_schema=False)
def page():
    return FileResponse(HERE / "predict_page.html")
```

코드	처음 배운 곳
`class Student(BaseModel)` + `Field(ge=0, le=16)`	2장 — 하루 시간이 말이 안 되는 값(예: 30시간)은 422 로 막아요

코드	처음 배운 곳
`@app.post("/predict")`	2장
`p = predict(...)`	이번 장 2절 — AI 함수 부르기
`"합격" if p >= 0.5 else "불합격"`	파이썬 — 확률 50% 이상이면 합격
`FileResponse(... "predict_page.html")`	4장 1단계

▶ POST /predict → 200

```
{
  "probability": 0.932,
  "label": "합격"
}
```

▶ POST /predict → 200

```
{
  "probability": 0.018,
  "label": "불합격"
}
```

예제 파일 `fastapi/초급과정/06_AI_맛보기/predict_page.html`

predict_page.html

```
<!DOCTYPE html>
<!-- 6장 합격 예측 화면: 4장 3단계(POST 보내기)와 같은 방법으로 /predict 를 불러요 -->
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>합격 예측</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 480px; margin: 40px auto; padding: 0 16px; }
    label { display: block; margin: 10px 0 4px; }
    input { padding: 8px; font-size: 15px; width: 120px; }
    button { margin-top: 14px; padding: 8px 16px; font-size: 15px; }
    #result { margin-top: 16px; padding: 14px; border-radius: 8px; font-size: 18px; min-height: 24px; }
    .pass { background: #e2f0d9; color: #385723; }
    .fail { background: #fce4d6; color: #843c0c; }
  </style>
</head>
<body>
  <h1>AI 합격 예측</h1>
  <p>하루 생활 시간을 넣으면, 학습한 모델이 합격 확률을 알려 줘요.</p>

  <form id="predict-form">
    <label>공부 시간 <input id="study" type="number" step="0.5" value="4"></label>
    <label>수면 시간 <input id="sleep" type="number" step="0.5" value="7"></label>
    <label>휴대폰 시간 <input id="phone" type="number" step="0.5" value="2"></label>
    <button type="submit">예측하기</button>
  </form>
  <div id="result"></div>

  <script>
    document.getElementById("predict-form").onsubmit = async (event) => {
      event.preventDefault();                                  // 페이지 새로고침 막기
      // 입력 칸의 값(글자) → 숫자로 바꿔 JSON 본문 만들기
      const body = {
        study_h: Number(document.getElementById("study").value),
        sleep_h: Number(document.getElementById("sleep").value),
        phone_h: Number(document.getElementById("phone").value),
```

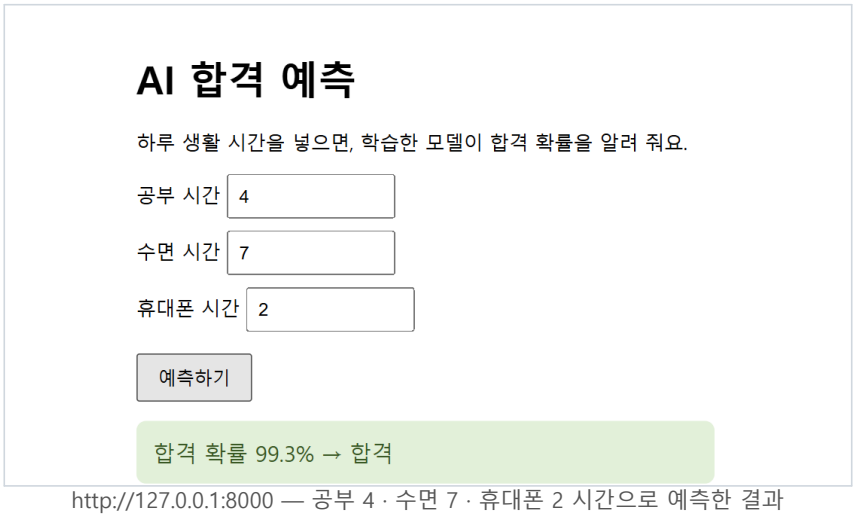
predict_page.html

```
      };
      // POST /predict 요청 보내기
      const res = await fetch("/predict", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(body),
      });
      const data = await res.json();
      const box = document.getElementById("result");
      if (res.ok) {
        const percent = (data.probability * 100).toFixed(1);        // 0.932 → "93.2"
        box.className = data.label === "합격" ? "pass" : "fail";
        box.textContent = `합격 확률 ${percent}% → ${data.label}`;
      } else {
        box.className = "fail";                                     // 422: 범위를 벗어난 값
        box.textContent = `입력 확인: ${data.detail[0].loc.at(-1)} — ${data.detail[0].msg}`;
      }
    };
  </script>
</body>
</html>
```

이 화면은 4장 3단계(학점 계산기)와 거의 같아요. 보내는 값과 결과를 보여 주는 문장만 달라요.

직접 해 보기 | 예측 서비스 써 보기

- `cd 06_AI_맛보기` → `uvicorn ex02_predict_app:app --reload` → `http://127.0.0.1:8000`
- 공부 시간을 1, 2, 3 으로 바꿔 가며 [예측하기] 를 눌러 보세요.
- 공부 시간에 30 을 넣으면 Pydantic 이 422 로 막고, 화면에 이유가 나와요.
- `http://127.0.0.1:8000/docs` 에서 POST /predict 를 Try it out 해도 같은 결과가 나와요.



http://127.0.0.1:8000 — 공부 4 · 수면 7 · 휴대폰 2 시간으로 예측한 결과

알아 두기 | 진짜 AI 서비스도 모양은 같아요

사진을 보고 고양이인지 맞히는 AI, 문장을 번역하는 AI 도 웹 서비스로 만들 때는 **"받은 값을 AI 함수에 넣고, 결과를 JSON 으로 돌려준다"** 는 모양이 똑같아요. 함수 안쪽만 더 크고 복잡할 뿐이에요. 더 알고 싶다면 부록 D 의 "심화 과정 6장" 을 보세요.

4. 정리

배운 것	한 줄 정리
AI 모델	학습해 둔 "잘 맞히는 함수". 우리는 불러다 쓰기만 해요
쓰는 법	`from ai_model import predict` → `predict(값들)`
예측 API	2장과 같아요: `BaseModel` 로 받고 → 함수에 넣고 → JSON 으로
예측 화면	4장 3단계와 같아요: 입력 칸 → `fetch` POST → 결과 표시

연습문제

문제 1. 공부 시간 0 ~ 8시간(1시간 간격)일 때의 합격 확률(%)을 `GET /table` 로 한꺼번에 돌려주세요. 수면 7시간, 휴대폰 2시간으로 고정해요.

문제 2. 예측 결과에 조언(`advice`)을 붙이세요. 합격 확률 50% 미만이면 "공부 시간을 조금 늘려 보세요", 80% 이상이면 "지금처럼만 하세요!", 그 사이면 "조금만 더 힘내요".

연습문제 정답 및 해설

정답 파일은 연습문제_정답 폴더에 있어요. ai_model.py 가 장 폴더에 있어서, **장 폴더(06_AI_맛보기)에서** uvicorn 연습문제_정답.문제1:app --reload 처럼 켜요.

문제 1 정답

문제1.py

```python
# 문제 1. 공부 시간 0 ~ 8시간(1시간 간격)일 때의 합격 확률을 GET /table 로 한꺼번에 돌려주세요.
#         수면 7시간, 휴대폰 2시간으로 고정해요.
#         예) /table → [{"study_h": 0, "probability": 1.1}, {"study_h": 1, ...}, ...]   (확률은 % 로)
# 실행: 장 폴더(06_AI_맛보기)에서  uvicorn 연습문제_정답.문제1:app --reload  →  /table
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent.parent))   # 장 폴더의 ai_model.py 를 찾을 수 있게
from fastapi import FastAPI

from ai_model import predict

app = FastAPI()


@app.get("/table")
def table():
    return [{"study_h": h, "probability": round(predict(h, 7, 2) * 100, 1)} for h in range(0, 9)]
```

예쁜 인쇄 ☐


```
[{"study_h":0,"probability":1.2},{"study_h":1,"probability":11.0},
{"study_h":2,"probability":56.6},{"study_h":3,"probability":93.2},
{"study_h":4,"probability":99.3},{"study_h":5,"probability":99.9},
{"study_h":6,"probability":100.0},{"study_h":7,"probability":100.0},
```


/table — 공부 시간별 합격 확률(%)

sys.path.insert(...) 줄은 "한 단계 위 폴더(장 폴더)의 파일도 찾아 줘" 라는 뜻이에요. 장 폴더에 바로 파일을 만들었다면 필요 없어요.

문제 2 정답

문제2.py

```python
# 문제 2. 예측 결과에 조언(advice)을 붙여 주세요.
#         합격 확률 50% 미만 → "공부 시간을 조금 늘려 보세요", 80% 이상 → "지금처럼만 하세요!", 그 사이 → "조금만 더 힘내요"
# 실행: 장 폴더(06_AI_맛보기)에서  uvicorn 연습문제_정답.문제2:app --reload  →  /docs 에서 POST /predict
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent.parent))   # 장 폴더의 ai_model.py 를 찾을 수 있게
from fastapi import FastAPI
from pydantic import BaseModel, Field

from ai_model import predict

app = FastAPI()


class Student(BaseModel):
    study_h: float = Field(ge=0, le=16)
    sleep_h: float = Field(ge=0, le=16)
    phone_h: float = Field(ge=0, le=24)
```

문제2.py

```
20
21
22 @app.post("/predict")
23 def predict_api(s: Student):
24     p = predict(s.study_h, s.sleep_h, s.phone_h)
25     if p < 0.5:
26         advice = "공부 시간을 조금 늘려 보세요"
27     elif p >= 0.8:
28         advice = "지금처럼만 하세요!"
29     else:
30         advice = "조금만 더 힘내요"
31     return {"probability": round(p, 3), "advice": advice}
```

▶ POST /predict → 200

```
{
  "probability": 0.566,
  "advice": "조금만 더 힘내요"
}
```

07장

미니 프로젝트

배운 것을 조립해 나만의 웹 서비스를 만들고, 친구 폰에서 접속해 발표해요

항목	내용
예제 폴더	`fastapi/초급과정/07_미니_프로젝트`
예제 파일	`guestbook/` (방명록 예시), `my_project/` (템플릿 + 기획서)
예상 시간	6시간

알아 두기 | 이번 장을 마치면 할 수 있어요

- 주제를 정하고 데이터와 API 를 설계할 수 있어요.
- 데이터를 JSON 파일에 저장해 서버를 꺼도 남게 할 수 있어요.
- 같은 와이파이의 휴대폰에서 내 서버에 접속할 수 있어요.
- 만든 서비스를 시연하고 발표할 수 있어요.

도움말 | 이번 장의 구성

- 1. 미니 프로젝트 진행 순서
- 2. 예시 프로젝트 — 우리 반 방명록
- 3. 친구 폰에서 내 서버에 접속하기
- 4. 내 프로젝트 만들기 — 템플릿에서 시작
- 5. 발표와 다음 단계

1. 미니 프로젝트 진행 순서

지금까지 배운 것만으로 **나만의 작은 웹 서비스**를 만들어 발표해요. 새 문법을 배우기보다, 배운 조각을 **내 아이디어에 맞게 조립하는** 연습이에요.

순서	할 일	쓰는 것
① 주제 정하기	누가 무엇을 하는 서비스인지 한 줄로	`my_project/기획서.md`
② 데이터 정하기	무엇을 저장할지 (이름 · 자료형 · 규칙)	2장 `BaseModel` · `Field`
③ API 설계	어떤 주소로 무엇을 주고받을지 표로	3장 CRUD
④ 서버 만들기	설계표대로 API 를 만들고 `/docs` 로 시험	1~3장
⑤ 화면 만들기	입력 칸 · 버튼 · 목록	4~5장
⑥ 발표	시연 + 친구 폰에서 접속해 보기	이 장 3절

주제 예시	저장할 데이터	난이도
우리 반 방명록 (2절 예시)	이름, 남길 말, 시각	★
맛집 / 책 / 영화 추천 목록	이름, 한 줄 평, 별점(1~5)	★
단어장 · 퀴즈	단어, 뜻, 맞힌 횟수	★★
용돈 기입장	날짜, 내용, 금액, 수입/지출	★★
6장 AI 예측 꾸미기	예측 기록 저장, 조언 문구, 화면 디자인	★★

2. 예시 프로젝트 — 우리 반 방명록

방명록으로 전체 흐름을 한 번 보고, 내 프로젝트를 시작해요. 5장 메모장과 거의 같고, 딱 하나가 새로워요. **글을 파일(posts.json)에 저장해서 서버를 꺼도 남아 있어요.**

예제 파일 `fastapi/초급과정/07_미니_프로젝트/guestbook/main.py`

main.py

```python
# 7장 미니 프로젝트 예시: 방명록 – 서버를 껐다 켜도 글이 남아 있어요 (posts.json 파일에 저장)
# 실행: 이 폴더(guestbook)에서  uvicorn main:app --reload
# 확인: 브라우저에서 http://127.0.0.1:8000
# 친구 폰에서 접속: uvicorn main:app --host 0.0.0.0  → 친구 폰 브라우저에 http://내컴퓨터IP:8000 (교안 7장 3절)
import json
from datetime import datetime
from pathlib import Path

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from pydantic import BaseModel, Field

app = FastAPI(title="방명록")
HERE = Path(__file__).parent
DATA_FILE = HERE / "posts.json"              # 글을 저장할 파일


# ── 파일 읽기 · 쓰기 (0장 4절의 JSON) ──
def load_posts():
    if DATA_FILE.exists():
        return json.loads(DATA_FILE.read_text(encoding="utf-8"))
    return []                                # 파일이 없으면(처음) 빈 목록


def save_posts(posts):
    DATA_FILE.write_text(json.dumps(posts, ensure_ascii=False, indent=2), encoding="utf-8")


# ── 받을 데이터 모양 (2장) ──
class PostIn(BaseModel):
    name: str = Field(min_length=1, max_length=10)        # 이름 1 ~ 10글자
    message: str = Field(min_length=1, max_length=100)    # 남길 말 1 ~ 100글자


# ── API (3장) ──
@app.get("/api/posts")
def list_posts():
    return list(reversed(load_posts()))       # 최신 글이 위로 오게 뒤집어서


@app.post("/api/posts", status_code=201)
def write_post(post: PostIn):
    posts = load_posts()
    new_id = posts[-1]["id"] + 1 if posts else 1          # 마지막 번호 + 1
    new_post = {"id": new_id, "name": post.name, "message": post.message,
                "time": datetime.now().strftime("%m-%d %H:%M")}
    posts.append(new_post)
    save_posts(posts)                         # 파일에 저장 → 서버를 꺼도 남아요
    return new_post


@app.delete("/api/posts/{post_id}")
def delete_post(post_id: int):
```

main.py

```
    posts = load_posts()
    left = [p for p in posts if p["id"] != post_id]       # 그 번호만 빼고 남기기
    if len(left) == len(posts):
        raise HTTPException(status_code=404, detail=f"{post_id}번 글이 없어요")
    save_posts(left)
    return {"message": f"{post_id}번 글을 지웠어요"}


# ── 화면 (4장) ──
@app.get("/", include_in_schema=False)
def page():
    return FileResponse(HERE / "index.html")
```

새로 나온 것	뜻
`load_posts()`	`posts.json` 파일을 읽어 리스트로 (파일이 없으면 빈 리스트)
`save_posts(posts)`	리스트를 `posts.json` 파일에 JSON 으로 쓰기 (0장 4절의 `json.dumps`)
저장 · 삭제 때마다 `save_posts`	딕셔너리(메모리) 대신 파일에 쓰니까 서버를 꺼도 남아요
`reversed(...)`	리스트를 뒤집기 — 최신 글이 위로
`datetime.now().strftime(...)`	지금 시각을 `11-06 14:30` 같은 글자로

예제 파일 `fastapi/초급과정/07_미니_프로젝트/guestbook/index.html`

index.html

```
<!DOCTYPE html>
<!-- 방명록 화면: 이름과 남길 말을 쓰면 POST /api/posts, 목록은 GET /api/posts (5장 메모장과 같은 방법) -->
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <!-- viewport: 휴대폰에서도 글자가 작지 않게 화면 폭에 맞추기 -->
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>우리 반 방명록</title>
  <style>
    body { font-family: "Malgun Gothic", sans-serif; max-width: 520px; margin: 30px auto; padding: 0 16px;
background: #f3f6fb; }
    h1 { text-align: center; }
    form { background: white; padding: 14px; border-radius: 10px; display: grid; gap: 8px; }
    input, textarea { padding: 8px; font-size: 15px; font-family: inherit; }
    button { padding: 8px; font-size: 15px; background: #2e75b6; color: white; border: none;
border-radius: 6px; }
    .post { background: white; padding: 12px 14px; border-radius: 10px; margin-top: 10px; }
    .post b { color: #1f4e79; }
    .post small { color: #888; margin-left: 6px; }
    .post p { margin: 6px 0 0; white-space: pre-line; }
    .post .del { float: right; background: none; color: #bbb; padding: 0; font-size: 13px; }
    #msg { color: #c00000; min-height: 18px; margin: 0; }
  </style>
</head>
<body>
  <h1>우리 반 방명록</h1>

  <form id="post-form">
    <input id="name" placeholder="이름 (10자까지)">
    <textarea id="message" rows="3" placeholder="남길 말 (100자까지)"></textarea>
    <button type="submit">남기기</button>
    <p id="msg"></p>
  </form>

  <div id="posts"></div>
```

index.html

```
34
35    <script>
36      // 글 목록 그리기 — GET /api/posts
37      async function loadPosts() {
38        const res = await fetch("/api/posts");
39        const posts = await res.json();
40        const box = document.getElementById("posts");
41        box.innerHTML = "";
42        for (const p of posts) {
43          const div = document.createElement("div");
44          div.className = "post";
45          div.innerHTML = `<button class="del">삭제</button><b></b><small></small><p></p>`;
46          div.querySelector("b").textContent = p.name;          // textContent: 글자를 그대로 넣기 (안전)
47          div.querySelector("small").textContent = p.time;
48          div.querySelector("p").textContent = p.message;
49          div.querySelector(".del").onclick = () => deletePost(p.id);
50          box.appendChild(div);
51        }
52      }
53
54      // 글 남기기 — POST /api/posts
55      document.getElementById("post-form").onsubmit = async (event) => {
56        event.preventDefault();
57        const res = await fetch("/api/posts", {
58          method: "POST",
59          headers: { "Content-Type": "application/json" },
60          body: JSON.stringify({
61            name: document.getElementById("name").value,
62            message: document.getElementById("message").value,
63          }),
64        });
65        if (res.ok) {
66          document.getElementById("message").value = "";
67          document.getElementById("msg").textContent = "";
68          loadPosts();
69        } else {
70          document.getElementById("msg").textContent = "이름(1~10자)과 남길 말(1~100자)을 확인해 주세요";
71        }
72      };
73
74      // 글 지우기 — DELETE /api/posts/번호
75      async function deletePost(id) {
76        if (!confirm("이 글을 지울까요?")) return;
77        await fetch("/api/posts/" + id, { method: "DELETE" });
78        loadPosts();
79      }
80
81      loadPosts();
82    </script>
83  </body>
84  </html>
```

화면 코드는 5장 메모장과 같은 모양이에요(목록 그리기, POST 로 저장, DELETE 로 삭제). `<meta name="viewport" ...>` 한 줄은 **휴대폰에서도 글자가 작지 않게** 화면 폭에 맞추는 설정이에요.

▶ 방명록 API 에 차례로 보낸 요청과 응답

```
POST /api/posts  →  201
  {"id": 1, "name": "철수", "message": "FastAPI 수업 재밌어요!", "time": "11-06 11:55"}
POST /api/posts  →  201
  {"id": 2, "name": "영희", "message": "방명록 잘 만들었다", "time": "11-06 11:55"}
GET /api/posts  →  200
```

▶ 방명록 API 에 차례로 보낸 요청과 응답

```
  [{"id": 2, "name": "영희", "message": "방명록 잘 만들었다", "time": "11-06 11:55"}, {"id": 1, "name": "철수",
"message": "FastAPI 수업 재밌어요!", "time": "11-06 11:55"}]
POST /api/posts  →  422
  {"detail": [{"type": "string_too_short", "loc": ["body", "name"], "msg": "String should have at least 1
character", "input": "", "ctx": {"min_length": 1}}]}
DELETE /api/posts/99  →  404
  {"detail": "99번 글이 없어요"}
```



http://127.0.0.1:8000 — 글 세 개를 남긴 방명록 (최신 글이 위)

직접 해 보기 | 방명록 써 보기

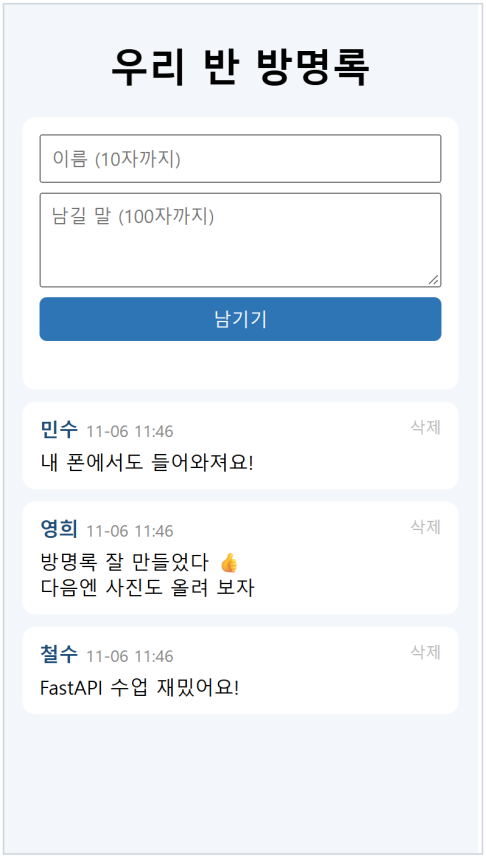
- `cd 07_미니_프로젝트/guestbook` → `uvicorn main:app --reload` → `http://127.0.0.1:8000`
- 글을 몇 개 남기고, 폴더에 `posts.json` 파일이 생긴 것을 확인해요. VS Code 로 열어 보면 남긴 글이 JSON 으로 들어 있어요.
- 서버를 `Ctrl + C` 로 끄고 다시 켠 뒤 새로고침해 보세요. **글이 그대로 남아 있어요!**

3. 친구 폰에서 내 서버에 접속하기

지금까지는 내 컴퓨터(`127.0.0.1`)에서만 접속했어요. 같은 와이파이에 연결된 **친구의 휴대폰에서 내 서버에 접속**해 볼게요. 내가 만든 서비스를 다른 사람이 쓰는 첫 경험이에요!

직접 해 보기 | 같은 와이파이의 휴대폰으로 접속하기

- 서버를 `uvicorn main:app --host 0.0.0.0` 로 켜요. `--host 0.0.0.0` 은 "다른 기기의 접속도 받을게요" 라는 뜻이에요.
- Windows 방화벽 창이 뜨면 **[허용]** 을 눌러요.
- 터미널 2 에서 `ipconfig` (macOS 는 `ipconfig getifaddr en0`) 를 입력해 **IPv4 주소**를 찾아요. 예: `192.168.0.15`
- 휴대폰을 **같은 와이파이**에 연결하고, 브라우저 주소창에 `http://192.168.0.15:8000` 처럼 입력해요.
- 휴대폰에서 글을 남기면 내 컴퓨터 화면에도 나타나요(새로고침). 같은 서버, 같은 `posts.json` 을 쓰니까요.



휴대폰 크기 화면으로 본 방명록 — viewport 설정 덕분에 글자가 작지 않아요

주의 | 접속이 안 되면

- 휴대폰과 컴퓨터가 **같은 와이파이**인지 확인해요 (휴대폰 데이터·다른 공유기는 안 돼요).
- 학교·회사 와이파이는 기기끼리 접속을 막아 둔 경우가 많아요. 그럴 땐 휴대폰 **핫스팟**에 컴퓨터를 연결해서 해 보세요.
- 방화벽 창에서 [취소] 를 눌렀다면, Windows 검색 → "방화벽을 통해 앱 허용" 에서 Python 을 허용해요.
- 수업이 끝나면 `--host 0.0.0.0` 없이 켜는 습관을 들여요. 다른 사람이 내 서버에 들어올 수 있으니까요.

4. 내 프로젝트 만들기 — 템플릿에서 시작

`my_project` 폴더에 **바로 실행되는 뼈대**가 있어요. 별점이 있는 목록을 저장하는 가장 단순한 서비스예요. 기획서를 먼저 채우고, 뼈대의 TODO 를 하나씩 내 주제에 맞게 바꿔요.

예제 파일 `fastapi/초급과정/07_미니_프로젝트/my_project/기획서.md`

기획서.md

```
# 미니 프로젝트 기획서

## 1. 서비스 소개
- 서비스 이름 :
- 한 줄 소개 : (누가, 무엇을 할 수 있는 서비스인가요?)
- 만든 사람 :

## 2. 저장할 데이터 (main.py 의 `ItemIn`)
| 항목 | 자료형 | 규칙 | 예 |
|---|---|---|---|
| title | str | 1 ~ 30글자 | 학교 앞 떡볶이 |
| score | int | 1 ~ 5 | 4 |
|  |  |  |  |
```

기획서.md

```
14
15 ## 3. API 설계표
16 | 하는 일 | 메서드 | 주소 | 보내는 것 | 돌려받는 것 |
17 |---|---|---|---|---|
18 | 목록 보기 | GET | /api/items | 없음 | 목록 |
19 | 저장하기 | POST | /api/items | title, score | 저장된 항목 |
20 |  |  |  |  |  |
21
22 ## 4. 화면 스케치
23 (종이에 그린 화면을 사진으로 붙여도 좋아요)
24
25 ## 5. 발표 체크리스트
26 - [ ] 서버를 켜고 브라우저에서 시연할 수 있어요
27 - [ ] `/docs` 에서 API 를 하나 이상 시험해 보일 수 있어요
28 - [ ] 이상한 값을 넣으면 422 로 막히는 것을 보여 줄 수 있어요
29 - [ ] 친구 폰에서 접속해 볼 수 있어요 (선택)
30 - [ ] 더 만들고 싶은 기능 한 가지를 말할 수 있어요
```

예제 파일 `fastapi/초급과정/07_미니_프로젝트/my_project/main.py`

main.py

```
1  # 나의 미니 프로젝트 — 이 파일을 고쳐서 나만의 웹 서비스를 만들어요
2  # 실행: 이 폴더(my_project)에서  uvicorn main:app --reload  →  http://127.0.0.1:8000
3  #
4  # 만드는 순서 (교안 7장 4절)
5  #   1) 기획서(기획서.md)에 서비스 이름과 "API 설계표" 를 먼저 채워요
6  #   2) 아래 TODO ① ~ ④ 를 설계표대로 바꿔요
7  #   3) /docs 에서 API 를 하나씩 시험해요
8  #   4) index.html 화면을 꾸며요
9  import json
10 from pathlib import Path
11
12 from fastapi import FastAPI
13 from fastapi.responses import FileResponse
14 from pydantic import BaseModel, Field
15
16 app = FastAPI(title="나의 미니 프로젝트")     # TODO ①: 서비스 이름으로 바꾸기
17 HERE = Path(__file__).parent
18 DATA_FILE = HERE / "data.json"
19
20
21 def load_items():
22     if DATA_FILE.exists():
23         return json.loads(DATA_FILE.read_text(encoding="utf-8"))
24     return []
25
26
27 def save_items(items):
28     DATA_FILE.write_text(json.dumps(items, ensure_ascii=False, indent=2), encoding="utf-8")
29
30
31 # TODO ②: 저장할 데이터의 모양을 정해요 (예: 맛집이면 이름·메뉴·별점)
32 class ItemIn(BaseModel):
33     title: str = Field(min_length=1, max_length=30)
34     score: int = Field(default=3, ge=1, le=5)
35
36
37 # TODO ③: 목록 보기 API — 필요하면 정렬·검색을 더해요
38 @app.get("/api/items")
39 def list_items():
40     return load_items()
```

main.py

```
41
42
43 # TODO ④: 저장 API — 받은 값에 번호를 붙여 파일에 저장해요
44 @app.post("/api/items", status_code=201)
45 def add_item(item: ItemIn):
46     items = load_items()
47     new = {"id": len(items) + 1, **item.model_dump()}   # model_dump(): 받은 값을 딕셔너리로
48     items.append(new)
49     save_items(items)
50     return new
51
52
53 # 화면
54 @app.get("/", include_in_schema=False)
55 def page():
56     return FileResponse(HERE / "index.html")
```

`my_project/index.html` 은 입력 칸 두 개와 목록이 있는 뼈대 화면이에요. `main.py` 의 `ItemIn` 칸을 바꾸면 화면의 입력 칸과 `JSON.stringify({...})` 안의 이름도 똑같이 바꿔 줘요.

▶ 뼈대 그대로 실행해 본 결과

```
POST /api/items  →  201
  {"id": 1, "title": "학교 앞 떡볶이", "score": 5}
POST /api/items  →  201
  {"id": 2, "title": "편의점 김밥", "score": 3}
GET /api/items  →  200
  [{"id": 1, "title": "학교 앞 떡볶이", "score": 5}, {"id": 2, "title": "편의점 김밥", "score": 3}]
```

도움말 | 막히면 이렇게 해 보세요

- **API 먼저, 화면은 나중에** — `/docs` 에서 API 가 잘 되는 걸 확인한 뒤 화면을 만들어요. 화면이 이상할 때 서버 문제인지 화면 문제인지 나눌 수 있어요.
- **한 번에 하나씩** — 기능 하나를 고치고, 저장하고, 확인하고. 여러 개를 한꺼번에 고치면 어디서 틀렸는지 찾기 어려워요.
- 브라우저에서 **F12 → Console** 탭을 열면 자바스크립트 오류가 빨간 글씨로 보여요. 서버 오류는 uvicorn 터미널에 나와요.
- 오류 메시지는 부록 C 에서 찾아봐요.

5. 발표와 다음 단계

알아 두기 | 발표 체크리스트 (기획서 5번)

- 서버를 켜고 브라우저에서 시연할 수 있어요
- `/docs` 에서 API 를 하나 이상 시험해 보일 수 있어요
- 이상한 값을 넣으면 422 로 막히는 것을 보여 줄 수 있어요
- 친구 폰에서 접속해 볼 수 있어요 (선택)
- 더 만들고 싶은 기능 한 가지를 말할 수 있어요

축하해요! 웹 API 를 만들고, 화면을 붙이고, 데이터를 저장하고, 다른 기기에서 접속하는 웹 서비스의 기본 흐름을 모두 경험했어요. 여기서 더 나아가는 길은 부록 D 에 정리해 두었어요. 데이터베이스, 로그인, 인터넷에 올리기(배포)처럼 궁금한 것부터 골라 스스로 공부해 보세요.

부록

부록 A. 용어 사전

용어	뜻	처음 나오는 곳
클라이언트 / 서버	요청을 보내는 쪽 / 요청을 받아 응답하는 쪽	0장 1절
요청 / 응답	클라이언트가 보내는 부탁 / 서버가 돌려주는 답	0장 1절
API	프로그램끼리 정해진 방식으로 데이터를 주고받는 창구	0장 1절
URL	요청을 보낼 주소. 호스트 + 포트 + 경로 + 쿼리	0장 2절
localhost / 127.0.0.1	내 컴퓨터 자신을 가리키는 주소	0장 2절
포트	한 컴퓨터 안에서 서버 프로그램을 구분하는 번호 (기본 8000)	0장 2절
HTTP 메서드	요청의 종류: GET 가져오기, POST 보내기, PUT 고치기, DELETE 지우기	0장 3절
상태 코드	응답 결과 번호: 200 성공, 201 만들기 성공, 404 없음, 422 형식 틀림	0장 3절
JSON	데이터를 주고받는 글자 형식. 파이썬 딕셔너리와 비슷	0장 4절
가상환경 (venv)	과정 전용 패키지 상자	0장 6절
uvicorn	FastAPI 앱을 실제 웹 서버로 실행해 주는 프로그램	0장 7절
데코레이터 `@app.get`	"이 주소로 요청이 오면 아래 함수를 실행" 이라고 등록하는 표시	1장 1절
경로 매개변수	주소 안의 값 — `/menu/{number}`	1장 2절
쿼리 매개변수	주소 뒤 `?` 의 값 — `/add?a=3&b=5`	1장 3절
`/docs`	FastAPI 가 자동으로 만들어 주는 API 시험 화면	1장 4절
본문 (body)	POST · PUT 요청에 담아 보내는 데이터(JSON)	2장 1절
Pydantic / BaseModel	받을 데이터의 모양을 정하고 자동으로 검사하는 도구	2장 2절
Field	값의 범위 · 글자 수 규칙 (`ge`, `le`, `min_length` ...)	2장 3절
CRUD	만들기 · 읽기 · 고치기 · 지우기	3장 1절
HTTPException	404 같은 오류 응답을 보내는 방법	3장 3절
HTML	웹 페이지의 뼈대(제목 · 글 · 버튼)를 쓰는 언어	4장 2절
자바스크립트	웹 페이지의 움직임(버튼을 누르면 할 일)을 쓰는 언어	4장 3절
fetch	자바스크립트에서 서버에 요청을 보내는 함수	4장 3절
동기 / 비동기	끝날 때까지 멈춰 기다리기 / 기다리는 동안 다른 일 하기	4장 3절
AI 모델	데이터를 보고 학습한 "잘 맞히는 함수"	6장 1절

부록 B. 명령어와 코드 모음

하고 싶은 일	명령 · 코드
가상환경 만들기 / 켜기	`python -m venv .venv` / `.venv\Scripts\activate` (Windows)
패키지 설치	`pip install fastapi uvicorn`
서버 켜기	`uvicorn 파일이름:app --reload` (예제 폴더에서)
다른 포트로 켜기	`uvicorn 파일이름:app --reload --port 8001`
다른 기기에서 접속 받기	`uvicorn 파일이름:app --host 0.0.0.0`

하고 싶은 일	명령 · 코드
서버 끄기	서버 터미널에서 `Ctrl + C`
API 시험 화면	`http://127.0.0.1:8000/docs`
앱 만들기	`app = FastAPI()`
주소 만들기	`@app.get("/주소")` + `def 함수(): return {...}`
경로 · 쿼리 받기	`@app.get("/menu/{number}")` + `def f(number: int, lang: str = "ko")`
본문 받기	`class Item(BaseModel): ...` + `@app.post("/items")` + `def f(item: Item)`
값 규칙	`age: int = Field(ge=0, le=150)`
오류 응답	`raise HTTPException(status_code=404, detail="없어요")`
HTML 돌려주기	`return FileResponse("page.html")`
화면에서 GET	`const res = await fetch("/주소"); const data = await res.json();`
화면에서 POST	`fetch("/주소", {method: "POST", headers: {"Content-Type": "application/json"}, body: JSON.stringify(값)})`

부록 C. 오류 해결 모음

각 장의 "자주 막히는 곳" 표를 한곳에 모았어요. 오류 메시지의 일부로 찾아보세요.

장	증상	원인	해결
0장	`'python' 은(는) 내부 또는 외부 명령…이 아닙니다`	파이썬이 설치되지 않았거나 PATH에 없음	설치할 때 **Add python.exe to PATH** 체크, 또는 `py` 명령 사용
0장	`No module named 'fastapi'`	가상환경이 꺼진 터미널에서 실행	줄 앞에 `(.venv)` 가 있는지 확인 → 켜고 다시 실행
0장	`uvicorn` 을 찾을 수 없음	설치 안 됨 / 가상환경 꺼짐	`pip install uvicorn` 또는 `python -m uvicorn 파일:app`
0장	`Could not import module "ex03_first_server"`	다른 폴더에서 서버를 켬	`cd` 로 예제 폴더에 들어가서 실행
0장	`address already in use` / `10048`	8000번 방에 이미 다른 서버가 있음	켜 둔 서버를 `Ctrl+C` 로 끄기, 또는 `--port 8001`
0장	브라우저에 "사이트에 연결할 수 없음"	서버가 꺼져 있음	터미널 1 에 `Uvicorn running…` 이 보이는지 확인
1장	`Could not import module "ex01_hello"`	다른 폴더에서 서버를 켬	`cd 01_첫_API_만들기` 로 예제 폴더에 들어가서 켜기
1장	`Attribute "app" not found`	`파일:app` 의 app 이름이 코드와 다름	코드의 `app = FastAPI()` 이름 확인
1장	고쳤는데 그대로예요	저장을 안 했거나 `--reload` 없이 켬	Ctrl+S 저장, `--reload` 붙이기
1장	`{"detail":"Not Found"}`	주소 오타, 등록 안 한 주소	`/docs` 에서 등록된 주소 확인
1장	422	숫자 자리에 글자, 꼭 보낼 값을 빠뜨림	응답의 `loc`·`msg` 읽기
2장	브라우저 주소창에 `/join` 을 치면 `Method Not Allowed`	주소창은 항상 GET 요청	POST 는 `/docs` 의 Try it out 으로
2장	422 + `"type": "missing"`	꼭 보낼 값(기본값 없는 필드)을 빠뜨림	`loc` 에 적힌 필드를 넣어 보내기
2장	422 + `greater_than_equal` 등	Field 규칙을 어김	`msg` 를 읽고 값 고치기

장	증상	원인	해결
2장	JSON 칸에서 오류	쉼표·따옴표가 빠짐, 마지막에 쉼표	JSON 은 큰따옴표만, 마지막 항목 뒤에 쉼표 금지
3장	메모를 만들었는데 목록이 비어 있어요	`--reload` 로 코드를 저장하면 서버가 다시 켜지면서 딕셔너리가 비워져요	코드를 다 고친 뒤 메모를 만들기
3장	`UnboundLocalError: next_id`	함수 안에서 `global next_id` 를 빠뜨림	함수 첫 줄에 `global next_id`
3장	405 Method Not Allowed	주소는 맞는데 메서드가 다름 (DELETE 를 GET 으로)	`/docs` 에서 메서드 확인
3장	404 `{"detail":"Not Found"}`	주소 자체가 틀림 (우리가 쓴 detail 이 아님)	주소 철자 확인

부록 D. 스스로 더 공부하기

초급 과정을 마쳤다면 웹 서비스의 기본 흐름(API → 화면 → 저장 → 다른 기기 접속)은 모두 경험한 거예요. 더 나아가고 싶다면 저장소의 `fastapi/심화과정` 폴더에 있는 원래 교안(Word)과 예제로 이어 가 보세요. 아래 순서를 추천해요.

순서	공부할 것	심화과정 위치	왜 필요할까?
1	응답 모양 정하기, 중첩 모델, 나만의 검사 규칙	`02_Pydantic_요청과_응답`	더 복잡한 데이터와 규칙
2	PATCH, 필터 · 페이지, `APIRouter` 로 파일 나누기	`03_CRUD_API_만들기`	API 가 많아질 때 정리하는 법
3	`Depends`, API 키, `async`, `lifespan`, CORS	`04_의존성_비동기_수명주기`	여러 API 가 함께 쓰는 기능, 운영 설정
4	파일 · 이미지 업로드	`05_파일업로드와_이미지`	사진을 받는 서비스
5	모델 학습 · 저장 · 서빙, 이미지 분류	`06_머신러닝_모델_서빙`	6장 AI 맛보기의 안쪽
6	프로젝트 구조, 자동 테스트(pytest), Docker	`07_테스트와_배포`	협업과 배포

그다음 주제	한 줄 설명	찾아볼 곳
데이터베이스	JSON 파일 대신 SQLite · PostgreSQL 에 저장	FastAPI 공식 문서 "SQL (Relational) Databases"
로그인	아이디 · 비밀번호로 로그인하고 토큰으로 확인	공식 문서 "Security"
배포	내 컴퓨터가 아닌 인터넷 서버에서 24시간 실행	공식 문서 "Deployment"
화면 꾸미기	HTML · CSS 과정, 더 나아가 React · Vue	저장소의 `html_css` 과정

> **도움말 | 공식 문서 읽는 법**
> FastAPI 공식 문서(`fastapi.tiangolo.com`)는 예제가 짧고 친절해요. 이 교안에서 배운 단어(경로 매개변수, 쿼리, 요청 본문 …)가 문서의 목차 이름과 같으니, 궁금한 단어로 찾아 읽고 **예제를 복사해 직접 실행**해 보세요.